

Cloud Provider Analytics — Documento de diseño

Primera evaluación parcial · Diseño y fundación de datos Big Data · ITBA · 2.º cuatrimestre 2026 · Prof. Adj. Diego Mosquera Entrega: 28/09/2026 18:30

Versión	1.0
Fecha	2026-09-20
Equipo	4 integrantes
Repositorio	(completar con la URL al publicar)

Trazabilidad con la consigna

§	Sección	Punto del alcance (§5.2 de la consigna)
1	Resumen ejecutivo	—
2	Problema, usuarios, preguntas y objetivos medibles	1
3	Justificación de la necesidad de Big Data: las 5V	2
4	Inventario y perfil de fuentes	3
5	Arquitectura de alto nivel	4
6	Patrón arquitectónico: elección y descarte	5
7	Matriz requisito-componente y mapeo 5V → decisión	6
8	Diseño del Data Lake	7
9	Flujos de datos batch y streaming	8
10	Flujo batch de referencia en lógica MapReduce	9
11	Supuestos, riesgos, mitigaciones y decisiones abiertas	10
12	Estimación de esfuerzo, roles y recursos	11
13	Anexo — Evidencia de exploración de datos	12

1. Resumen ejecutivo

El problema. El área de datos de un proveedor de nube necesita publicar dato confiable para FinOps, Soporte y Producto. Hoy las siete fuentes batch y el flujo de eventos llegan crudos, con identificadores duplicados, tipos ambiguos, costos negativos, invariantes violadas y una evolución de esquema a mitad del histórico. La dificultad de fondo no es el volumen: son **dos latencias incompatibles** —minutos para detectar una anomalía de costo antes del cierre, un mes para consolidar la facturación— que ninguna ruta única resuelve bien.

La propuesta. Un Data Lake de cuatro zonas en Parquet (Landing → Bronze → Silver → Gold) procesado con PySpark, con serving query-first en Cassandra/AstraDB, bajo un **patrón híbrido**: rutas segmentadas por fuente según la latencia que su decisión de negocio exige, más un job batch de reconciliación que corrige lo que el watermark del streaming descarta.

La justificación. El dataset provisto pesa 2,46 MB y **ese volumen no justifica Big Data**. Lo que lo justifica es que sea una réplica reducida que conserva todas las características cualitativas del caso real —variedad estructural, veracidad comprometida y velocidad con desorden— y que el caso al que debe escalar supera los mil millones de eventos anuales. El volumen es una de cinco V, y acá no es la dominante.

Tres hallazgos del perfilado que gobernaron el diseño. Las cifras salen de ejecutar PySpark 4.2.0 sobre las ocho fuentes; el script es reproducible.

- La fragmentación de eventos es aleatoria, no temporal.** Cada archivo cubre 59 días sin orden cronológico. Con un watermark de 7 días, la agregación en streaming procesaría el 8,9% de los eventos; incluso con 30 días descartaría casi la mitad. La llegada tardía es estructural, y esto es lo que hace necesaria la reconciliación: no es una precaución teórica, es lo único que hace correctos a los marts.
- Los problemas de calidad más graves no son nulos.** El 35,1% de los usuarios con login tiene `last_login` anterior a `created_at`, y 96 interacciones de marketing registran conversión sin clic. Son invariantes entre columnas, con todos los campos completos y bien tipados: ningún control de completitud los detecta.
- Las reglas mal calibradas son peores que la ausencia de reglas.** Evaluar el NPS contra la escala 0-10 marcaba 59 de 80 registros como anómalos; con la escala correcta (−100 a +100) hay una sola violación. Escribir la regla de `genai_tokens` como "v2 implica tokens" en lugar de `v2 AND service = 'genai'` produciría 251 falsos positivos sobre 275 eventos.

Decisiones de diseño con consecuencia práctica. Bronze particiona por fecha de ingesta y Silver por fecha de evento, lo que evita el problema de small files y permite acotar la reconciliación a las particiones efectivamente tocadas. El rechazo por calidad opera sobre el campo derivado y no sobre el registro, porque descartar registros completos perdería el 4,6% del costo. La tasa de cambio se fuerza a 1,0 para facturas en USD: el ruido se compensa en el agregado (0,1% del revenue total) pero distorsiona hasta ±15% por factura, y los marts reportan por organización. Las anomalías se detectan con MAD sobre la serie diaria por organización y servicio, porque cualquier estadística global marca la métrica cara por ser cara —un evento de `requests` cuesta 87 veces la mediana de uno de `cpu_hours`—.

Estado y límites. Diez decisiones cerradas y registradas con su alternativa descartada; tres abiertas con fecha. La más relevante es **A-07**: el perfilado se ejecutó sobre 1 de los ~20 archivos de eventos, y si el conjunto completo mostrara llegada ordenada, caerían los fundamentos del watermark y posiblemente del patrón. El entorno declarado es Colab en modo local, con paralelismo simulado y almacenamiento efímero: el diseño de particiones se justifica contra el caso dimensionado, no contra el runtime de la demo.

Esfuerzo. 286 horas-persona para el proyecto completo, sobre un equipo de 4 integrantes con roles asignados por capacidad end-to-end y suplencias nombradas.

2. Problema, usuarios, preguntas y objetivos medibles

2.1 El problema

El área de datos de un proveedor de nube tiene que sostener decisiones de negocio sobre datos que hoy llegan crudos, dispersos en siete fuentes de maestros y hechos, más un flujo continuo de eventos de uso. Esos datos no están conformados: conviven identificadores duplicados, tipos ambiguos, nulos con significados distintos, costos negativos, invariantes temporales violadas y una evolución de esquema a mitad del histórico.

El problema **no es la ausencia de datos, sino la ausencia de un dato confiable, conformado y consultable con la latencia que cada decisión exige**. Hoy no se puede responder cuánto gastó una organización ayer sin cruzar manualmente fuentes que no comparten criterio de calidad.

Esa brecha se manifiesta en dos latencias incompatibles entre sí:

- **Operativa, de minutos.** El consumo y el costo incremental cambian de forma continua. Una anomalía de costo detectada al cierre del mes ya se facturó.
- **Analítica, de horas o días.** Los maestros de CRM, la facturación mensual y las encuestas se consolidan por lote y no requieren frescura inmediata.

Resolver una sola de las dos deja la mitad del negocio sin respuesta. Esta tensión es la que gobierna la decisión de arquitectura de la \$6.

2.2 Usuarios y decisiones que deben tomar

Un dominio se justifica por la decisión que habilita, no por el reporte que produce.

Dominio	Quién lo usa	Decisión que toma	Latencia tolerable
FinOps	Finanzas, Customer Success, cuentas enterprise	Intervenir un consumo anómalo antes del cierre; validar revenue con créditos, impuestos y FX; renegociar plan	Minutos para anomalías; mensual para revenue
Soporte	Jefatura de soporte, responsables de SLA	Reasignar capacidad ante picos de severidad; explicar incumplimientos de SLA; correlacionar CSAT con riesgo de churn	Horas
Producto / Usage	Product managers, equipo GenAI	Priorizar roadmap por adopción real de servicios; dimensionar costo e impacto de GenAI; reportar huella de carbono	Diaria

La consigna define esos tres dominios. La contribución del diseño es hacer explícita la decisión y la latencia de cada uno, porque son esas dos columnas —y no el nombre del dominio— las que determinan si la ruta es batch o streaming.

2.3 Preguntas principales

Las preguntas se numeran para poder trazarlas hasta los marts de la capa Gold y hasta las consultas obligatorias de la evaluación final (\$7.4 de la consigna).

#	Pregunta	Dominio	Consulta obligatoria
P1	¿Cuál fue el costo y el volumen de requests diarios por organización y servicio en un rango de fechas?	FinOps	1
P2	¿Cuáles son los N servicios de mayor costo acumulado en los últimos 14 días para una organización?	FinOps	2
P3	¿Cómo evolucionaron los tickets críticos y la tasa de incumplimiento de SLA en los últimos 30 días?	Soporte	3
P4	¿Cuál fue el revenue mensual por organización, neto de créditos, con impuestos y normalizado a USD?	FinOps	4
P5	¿Cuántos tokens GenAI se consumieron por día y cuál fue su costo estimado?	Producto	5
P6	¿Qué consumos se desvían de su comportamiento histórico y merecen revisión?	FinOps	—
P7	¿Qué relación hay entre la experiencia de soporte y el riesgo de abandono de una organización?	Soporte / CS	—

P1 a P5 son obligatorias y fijan el alcance mínimo. P6 corresponde al mart de anomalías que la consigna exige en la capa Gold. P7 es la pregunta que habilita el componente analítico o de ML, y es la única cuyo valor depende de datos que el perfilado ya mostró incompletos: 20 de 80 organizaciones nunca fueron encuestadas y el 25,4% de los tickets no tiene CSAT. Se declara deseable, no obligatoria.

2.4 Objetivos medibles

Un objetivo sin umbral no es verificable. Estos son los criterios de éxito de la plataforma, y se retoman como criterios de aceptación en las entregas siguientes.

#	Objetivo	Métrica	Umbral	Se verifica en
O1	Frescura de las métricas operativas	Latencia evento → mart consultable	≤ 5 min	2.ª entrega
O2	Completitud de la ruta batch	Maestros y facturación conformados por corrida	100% de las fuentes	2.ª entrega
O3	Calidad controlada y trazable	Registros inválidos aislados en quarantine, nunca descartados en silencio	100% de los inválidos con motivo	2.ª entrega
O4	Reprocesamiento sin duplicados	Conteo antes/después de re-ejecutar	Variación = 0	2.ª entrega
O5	Cobertura analítica	Marts que responden las preguntas obligatorias	P1 a P5	Final
O6	Latencia de consulta en serving	Tiempo de respuesta de las 5 consultas en Cassandra	< 1 s	Final
O7	Reproducibilidad	Ejecución completa desde entorno limpio siguiendo el Quickstart	Sin pasos manuales ocultos	Final
O8	Compatibilidad de esquema	Eventos v1 y v2 consultables en el mismo mart sin ruptura	100%	2.ª entrega

2.5 Alcance

Dentro: ingesta batch de los siete maestros y hechos, ingesta streaming de los eventos de uso, Data Lake de cuatro zonas en Parquet, marts Gold para FinOps, Soporte y Producto, serving en Cassandra/AstraDB con modelado query-first, controles de calidad con quarantine, detección de anomalías de costo, e idempotencia en ambas rutas.

Fuera: herramienta de visualización propia (el serving se expone para que la consuma un BI externo), orquestación productiva con Airflow, autenticación de usuarios finales, y optimización de costo de infraestructura real. Se documentan como próximos pasos en la \$12.

3. Justificación de la necesidad de Big Data: las 5V

3.1 Una aclaración previa, y por qué importa

El dataset provisto pesa **2,46 MB**: 4.112 filas en las siete fuentes batch y unos 7.200 eventos en los ~20 archivos de streaming. **Ese volumen no justifica Big Data**. Se procesa con pandas en una notebook y sobra.

Sostener lo contrario sería el primer error de criterio del proyecto, y la Clase 03 lo advierte explícitamente: usar Spark "por moda" produce notebooks lentos, costosos y difíciles de depurar. La decisión técnica madura empieza por elegir bien la herramienta.

La justificación correcta es otra, y tiene dos partes.

Primero, el dataset es una réplica reducida de una carga productiva, no la carga misma. Conserva deliberadamente todas las características cualitativas del caso real —variedad de formatos, evolución de esquema, tipos ambiguos, llegada tardía, anomalías— y reduce únicamente el volumen. Diseñar para el tamaño de la muestra y no para el del caso obliga a rehacer la arquitectura al primer crecimiento.

Segundo, el volumen es solo una de las cinco V, y no es la que gobierna este caso. Lo que exige un motor distribuido acá es la combinación de velocidad sostenida, variedad estructural y veracidad comprometida. Un dataset chico con esos tres problemas es un caso de Big Data mal dimensionado; un dataset grande y limpio muchas veces no lo es.

Esta distinción se declara como supuesto en la \$11 y es la premisa sobre la que se apoya toda la sección.

3.2 Volumen

En la muestra: 2,46 MB, 4.112 filas batch, ~7.200 eventos, ~300 bytes por evento.

En el caso real: los 400 recursos del dataset a lo largo de 60 días representan 576.000 recursos-hora. Los eventos provistos equivalen a **0,0125 eventos por recurso-hora**: apenas un 1,25% de lo que emitiría una telemetría que reportara una sola métrica por recurso y por hora. La muestra está deliberadamente diezmada.

Dimensionando el caso al que la plataforma debe poder crecer, con tres métricas por recurso-hora:

Escenario	Organizaciones	Recursos	Eventos/día	Eventos/año
Dataset provisto	80	400	~120	~44 mil
Telemetría completa del mismo dataset	80	400	28.800	~10,5 millones
Proveedor mediano	5.000	50.000	3,6 millones	~1.300 millones
Proveedor grande	50.000	2.000.000	144 millones	~52.600 millones

A 300 bytes por evento, el escenario mediano produce cerca de **400 GB anuales solo de eventos crudos**, antes de Bronze, Silver y Gold.

Decisión que fuerza: almacenamiento columnar particionado (Parquet) en lugar de archivos crudos, procesamiento distribuido con poda de particiones, y una estrategia de retención diferenciada por zona. Se desarrolla en la §8.

3.3 Velocidad

En la muestra: los eventos llegan fragmentados en ~20 archivos que simulan micro-lotes. El perfilado mostró que **cada archivo cubre 59 días y no está ordenado cronológicamente**: el primer registro es del 18/08 y el segundo del 12/08.

Eso es más exigente que un flujo ordenado. La fragmentación es aleatoria, no temporal, con lo que **la llegada tardía es la norma estructural y no una excepción de red**. Un pipeline que asuma orden razonable de llegada produce resultados incorrectos en cada micro-lote.

Hay además dos velocidades que conviven: eventos continuos frente a facturación que se consolida una vez por mes (month tiene exactamente 3 valores) y maestros que cambian a ritmo diario.

Decisión que fuerza: Structured Streaming con watermark dimensionado contra el desorden observado y no contra una latencia optimista; deduplicación por event_id sostenida en estado e **inter-lote**, no un dropDuplicates por micro-batch; y checkpointing para garantizar el objetivo O4. Es también el argumento central de la elección de patrón en la §6.

3.4 Variedad

En la muestra, tres ejes simultáneos:

- **De formato:** CSV tabular y JSONL semiestructurado, con esquemas y estrategias de lectura distintas.
- **Estructural dentro de una misma fuente:** schema_version 1 y 2 conviven en el mismo archivo, con 11 y 13 campos respectivamente. El corte es el 2025-07-18.
- **Anidada dentro de un campo:** tags_json es un array embebido como texto dentro de un CSV (["env:prod"]), nulo en el 20,75% de los recursos.

A esto se suma la variedad semántica: siete fuentes con granos distintos (organización, usuario, recurso, ticket, interacción, factura mensual, evento) que deben conformarse contra una única dimensión de organizaciones.

Decisión que fuerza: esquemas explícitos en toda lectura —nunca inferSchema, siguiendo la Clase 03— y una zona Silver cuya responsabilidad específica es la conformación: promover v1 al esquema ampliado con nulos explícitos, parsear tags_json a array, y normalizar regiones, servicios y unidades antes de cualquier join.

3.5 Veracidad

Es la V dominante de este caso. El perfilado documentó diez riesgos de calidad (§4.5), de los cuales cuatro merecen destacarse porque **ningún control de completitud los detecta**:

Hallazgo	Magnitud	Por qué es grave
last_login anterior a created_at	232 de 661 usuarios con login (35,1%)	Invariante temporal violada; los campos están completos y bien tipados
converted = true sin clicked = true	96 de 1.500 interacciones	Inconsistencia lógica del embudo entre dos columnas válidas
FX ruidoso sobre facturas en USD	160 de 160 filas USD, tasa en [0,855; 1,118]	Se compensa en el agregado (0,1%) pero distorsiona hasta ±15% por factura
Costos negativos de magnitud	mínimo −19,82 frente a un p99 de 15,35	No es un redondeo: es comparable al percentil 99 positivo

Y dos distinciones que condicionan el diseño de las reglas:

- **Nulos semánticos frente a defectos.** Los 240 resolved_at nulos son tickets abiertos y los 254 csat nulos son encuestas no respondidas: son estado válido. Enviarlos a quarantine rompería el mart de Soporte. En cambio los 17 unit nulos con value presente sí son defectos.
- **Reglas mal calibradas.** Evaluar nps_score contra la escala 0–10 marcaba 59 de 80 registros como anómalos; con la escala NPS correcta (−100 a +100) hay una sola violación. Y escribir la regla de genai_tokens como "v2 implica tokens" en lugar de v2 AND service = 'genai' produciría 251 falsos positivos sobre 275 eventos. Una regla que grita todo el tiempo se termina ignorando.

Decisión que fuerza: reglas verificables que incluyan **invariantes entre columnas** y no solo validaciones campo a campo; quarantine en Parquet con motivo de rechazo, sin descartes silenciosos; y flags de anomalía en lugar de eliminación, porque un costo negativo puede ser una nota de crédito legítima. La asimetría de la distribución de costos (media 2,59, mediana 0,74, p99 15,35) desaconseja z-score puro y favorece MAD o percentiles: es la decisión abierta A-04.

3.6 Valor

El valor se mide contra las decisiones de la §2.2, no contra el volumen procesado.

- **FinOps:** detectar una desviación de consumo antes del cierre en lugar de explicarla después. Con 6 eventos por encima de 3 desvíos en una muestra de 360, el escenario mediano implicaría decenas de miles de anomalías diarias a priorizar automáticamente.
- **Soporte:** 95 incumplimientos de SLA sobre 1.000 tickets (9,5%) y 56 tickets críticos son la base cuantitativa para dimensionar capacidad.
- **Producto:** los eventos GenAI aparecen recién a partir del 2025-07-18 y solo en el 6,7% de los eventos v2. Medir su adopción real y su costo asociado es exactamente la pregunta que un producto nuevo necesita responder.
- **Transversal:** la huella de carbono (carbon_kg , presente en el 100% de los eventos v2) pasa a ser reportable sin trabajo adicional.

Decisión que fuerza: marts Gold orientados a pregunta y no a fuente, con modelado query-first en Cassandra. La tabla se diseña desde la consulta que debe responder, que es la razón por la que P1–P5 se trazaron explícitamente en la §2.3.

3.7 Síntesis

V	Evidencia en la muestra	Decisión de arquitectura
Volumen	2,46 MB, pero 0,0125 eventos por recurso-hora: muestra diezmada de un caso que escala a miles de millones de eventos anuales	Parquet particionado, procesamiento distribuido, retención por zona
Velocidad	Micro-lotes con span de 59 días, sin orden cronológico	Structured Streaming, watermark, dedupe inter-lote con estado, checkpointing
Variedad	CSV + JSONL, schema_version 1 y 2 conviviendo, tags_json anidado	Esquemas explícitos, Silver como capa de conformación
Veracidad	10 riesgos documentados; 35% de invariantes temporales violadas; FX sobre USD	Reglas con invariantes entre columnas, quarantine con motivo, flags sobre descarte
Valor	Anomalías accionables, SLA cuantificado, adopción GenAI, carbono	Marts query-first trazados a P1-P5

Las cinco V se retoman en la §7, donde cada una se mapea contra el componente concreto que la resuelve.

4. Inventario y perfil de fuentes

4.1 Método

El perfilado se ejecutó con **Apache Spark 4.2.0 (PySpark, master=local[*])** sobre la zona Landing en modo estrictamente de lectura, respetando la inmutabilidad del dato crudo exigida por la consigna (§3.2). El script es notebooks/01_perfilado_fuentes.py y su salida completa está en evidence/perfilado_landing.md y evidence/perfilado_metrics.json. Todas las cifras de esta sección son reproducibles.

Para cada fuente se midió: esquema y tipos, volumen, nulos y vacíos por columna, cardinalidad exacta, dominios de los campos categóricos, rangos temporales, violaciones de invariantes de negocio e integridad referencial contra la dimensión de organizaciones.

Dos decisiones metodológicas vale la pena explicitar, porque corrigen errores que el perfilado inicial produjo:

- **Cardinalidad exacta, no aproximada.** approx_count_distinct opera sobre HyperLogLog y a esta escala introduce un error visible: reportaba 83 org_id distintos sobre 80 filas. Se usa count_distinct. A volúmenes de producción la decisión se revierte, y ese trade-off queda registrado en DECISIONS.md.
- **Los rangos válidos se derivan del dominio, no se asumen.** La primera pasada evaluó nps_score contra la escala 0-10 de una respuesta individual de encuesta y marcó 59 de 80 registros como anómalos. La distribución real (mín. -38, máx. 101, mediana 14) corresponde a la métrica **NPS agregada, cuya escala es -100 a +100**. Con la regla correcta hay **una sola** violación. Una regla de calidad mal calibrada no genera falsos positivos: genera desconfianza en todo el control de calidad.

4.2 Resumen de fuentes

Fuente	Filas	Cols.	Grano	Naturaleza	Frecuencia	Zona de destino
customers_orgs.csv	80	11	1 fila = organización	Dimensión (maestro)	Batch diario	Bronze → Silver (SCD)
users.csv	800	7	1 fila = usuario	Dimensión (maestro)	Batch diario	Bronze → Silver
resources.csv	400	7	1 fila = recurso cloud	Dimensión (maestro)	Batch diario	Bronze → Silver
support_tickets.csv	1.000	8	1 fila = ticket	Hecho (batch)	Batch diario	Bronze → Silver → Gold Soporte
marketing_touches.csv	1.500	7	1 fila = interacción	Hecho (batch)	Batch diario	Bronze → Silver
nps_surveys.csv	92	4	1 fila = encuesta org+fecha	Hecho (serie temporal)	Batch diario	Bronze → Silver
billing_monthly.csv	240	8	1 fila = factura org+mes	Hecho (batch)	Batch mensual	Bronze → Silver → Gold FinOps
usage_events_stream/*.jsonl	360 por archivo (~20 archivos)	11 (v1) / 13 (v2)	1 fila = evento de uso	Hecho (streaming)	Micro-lotes	Bronze → Silver → Gold FinOps/Producto

Ventana temporal del dataset: **2025-05-04 a 2025-08-31** para los maestros y hechos batch; **2025-07-03 a 2025-08-31** para los eventos de uso. La facturación cubre tres meses cerrados: 2025-06, 2025-07 y 2025-08.

4.3 Perfil por fuente

customers_orgs.csv — dimensión de organizaciones

80 filas, org_id único y sin duplicados. Es la dimensión conformada contra la que se valida el resto del modelo.

- **Dominios:** plan_tier = standard (41), pro (21), enterprise (10), free (8). lifecycle_stage = active (54), at_risk (11), churned (6), prospect (6), lead (3). hq_region = 7 regiones. industry = 10 valores.
- **Calidad:** nps_score nulo en 11 registros (13,75%) y una violación de rango (101 > 100). El resto de las columnas está completo.
- **Riesgo:** nps_score aparece tanto acá como en nps_surveys.csv, con valores que no coinciden. Es una redundancia que exige definir cuál es la fuente de verdad. Se resuelve en la §8.

users.csv — usuarios por organización

800 filas, user_id único, 80 organizaciones representadas.

- **Dominios:** role = data_engineer (201), devops (162), developer (130), analyst (113), ml_engineer (105), admin (89).
- **Calidad:** last_login nulo en 139 usuarios (17,38%), interpretable como usuario que nunca ingresó y no necesariamente como defecto.
- **Hallazgo relevante:** 232 de los 661 usuarios con login registrado (35,1%) tienen last_login anterior a created_at. Es una violación de invariante temporal, no un nulo, y no se detecta con un control de completitud. Justifica que las reglas de calidad incluyan **invariantes entre columnas**, no solo validaciones campo a campo.

resources.csv — recursos cloud

400 filas, resource_id único, 6 servicios y 7 regiones.

- **Dominios:** service = compute (116), storage (71), database (68), networking (64), analytics (42), genai (39). state = running (242), stopped (119), terminated (39).
- **Calidad:** tags_json nulo en 83 registros (20,75%). Es un campo **semiestructurado embebido en un CSV** (["env:prod"]), que requiere parseo a array en Silver. Aporta a la V de variedad.

support_tickets.csv — tickets de soporte

1.000 filas. ticket_id presenta 983 valores distintos: **17 identificadores repetidos**, que obligan a deduplicación en Bronze.

- **Dominios:** severity = low (412), medium (328), high (204), critical (56). category = 6 valores balanceados.
- **Calidad:** resolved_at nulo en 240 (24%) — tickets abiertos, estado legítimo. csat nulo en 254 (25,4%) — encuesta no respondida. 95 tickets con sla_breached = true (9,5%). Cero inversiones resolved_at < created_at.
- **Nota de diseño:** los nulos de resolved_at y csat son **semánticos, no defectos**. Enviarlos a quarantine sería un error: el mart de Soporte necesita contar tickets abiertos.

marketing_touches.csv — interacciones de marketing

1.500 filas. touch_id con 1.439 distintos: **61 duplicados**.

- **Dominios:** channel = event (401), email (385), ads (358), in_app (356). campaign = 6 campañas.
- **Calidad:** 96 registros con converted = true y clicked = false, inconsistencia lógica del embudo. Es la segunda invariante entre columnas del dataset.

nps_surveys.csv — encuestas NPS

92 filas sobre 60 organizaciones. **32 organizaciones tienen más de una encuesta**, con lo que la tabla es una serie temporal y no una dimensión: al conformar hay que decidir entre último valor vigente o SCD tipo 2.

- **Calidad:** nps_score nulo en 19 (20,65%), comment nulo en 10 (10,87%). Ninguna violación de la escala −100/+100. comment es texto categórico de 6 valores, no libre.
- **Cobertura:** 20 de las 80 organizaciones nunca fueron encuestadas — el join debe ser left, no inner.

billing_monthly.csv — facturación mensual

240 filas = 80 organizaciones × 3 meses, cobertura completa. invoice_id con 231 distintos: **9 duplicados**.

- **Calidad:** credits nulo en 137 (57,08%) — ausencia de crédito, debe resolverse como 0 y no como nulo propagado. 13 subtotales negativos (notas de crédito o ajustes).
- **Hallazgo crítico para FinOps:** currency = USD (160), ARS (51), EUR (29). Las tasas de ARS (mediana 0,00150) y EUR (mediana 1,10467) son plausibles, pero **las 160 filas en USD tienen tasas que oscilan entre 0,85463 y 1,11791 en lugar de ser exactamente 1,0**. Es ruido alrededor de 1, no una tasa real: convertir dólares a dólares tiene tasa 1 por definición.

El impacto merece precisión. **En el agregado el ruido se compensa y la diferencia de revenue total es del 0,1%**, con lo que una lectura global no lo detectaría. Pero **por factura individual la distorsión llega a −15% / +12%**, y los marts de FinOps reportan por organización y por mes, no en total. La regla de normalización debe condicionarse por moneda. Se resuelve en la §8.8.

usage_events_stream/*.jsonl — eventos de uso (fuente de streaming)

Perfilado sobre events_part_0104.jsonl (360 eventos), representativo del resto. El dataset completo son ~20 archivos.

Esquema leído con **declaración explícita** (nunca inferSchema, siguiendo la Clase 03), con value tipado como String de forma deliberada porque la fuente lo emite indistintamente como número o como texto.

Campo	Tipo	Presencia
event_id	string	siempre — clave natural de deduplicación
timestamp	string ISO-8601 UTC	siempre — event time
org_id, resource_id	string	siempre — FK a customers_orgs y resources
service, region, metric, unit	string	siempre salvo unit
value	string null	tipo ambiguo
cost_usd_increment	double	siempre
schema_version	int	1 o 2
carbon_kg	double	solo v2
genai_tokens	long	solo v2 y service = genai

Evolución de esquema. El corte es limpio y verificable:

schema_version	Eventos	Desde	Hasta	Con carbon_kg	Con genai_tokens
1	85	2025-07-03 05:32	2025-07-17 20:32	0	0
2	275	2025-07-18 04:15	2025-08-31 15:15	275	24

La condición de presencia de genai_tokens es schema_version = 2 AND service = 'genai', no solo la versión: aparece en 24 de 24 eventos que cumplen ambas, y en 0 eventos de otros servicios. Escribir la regla como "v2 implica tokens" produciría 251 falsos positivos. La compatibilidad v1/v2 se resuelve promoviendo v1 al esquema ampliado con nulos explícitos, distinguibles de un nulo por defecto de calidad.

Calidad: value nulo en 9 (2,5%) y unit nulo en 17 casos con value presente — esta última es literalmente una de las tres reglas de calidad que la consigna exige para la segunda entrega (§6.2). En esta muestra no hay event_id duplicados dentro del archivo, pero la deduplicación sigue siendo obligatoria por lo que se explica abajo.

Anomalías de costo: cost_usd_increment tiene media 2,5912 y desvío 4,0111, con p50 0,7426, p95 10,7290 y p99 15,3517. La distribución es marcadamente asimétrica. Hay 6 eventos con |z| > 3 y 2 por debajo de −0,01, con un mínimo de **−19,8245** — un negativo de magnitud comparable al p99 positivo, no un redondeo. La asimetría desaconseja z-score puro como único método de detección y favorece MAD o percentiles; la decisión se fundamenta en la §11.

Fragmentación de archivos — el hallazgo de diseño más importante de esta sección. El archivo analizado cubre **59 días**, prácticamente la ventana completa del dataset, y sus eventos no están ordenados cronológicamente: el primer registro es del 18/08 y el segundo del 12/08. La fragmentación en ~20 archivos es **aleatoria, no temporal**.

Las consecuencias son directas sobre la arquitectura:

1. Cada micro-lote traerá eventos antiguos y recientes mezclados, con lo que **la llegada tardía es la norma y no la excepción**. El watermark debe dimensionarse contra el span observado y no contra una latencia de red optimista.
 2. La deduplicación por event_id debe ser **inter-lote**, sostenida en estado con watermark, y no un dropDuplicates dentro de cada micro-batch.
 3. El particionado de Bronze por fecha de evento implica que **cada micro-lote escribe en casi todas las particiones**, lo que genera small files y obliga a una estrategia de compactación.
- Esto se desarrolla en las §8 y §9.

4.4 Integridad referencial

Fuente	Orgs distintas	org_id sin match en customers_orgs
users	80	0
resources	80	0
support_tickets	80	0
marketing_touches	80	0
nps_surveys	60	0
billing_monthly	80	0
usage_events	78	0

Cero huérfanos en todas las fuentes, y 0 `resource_id` de eventos sin match en `resources.csv` sobre 228 distintos. La integridad referencial se sostiene, lo que habilita `left join` contra la dimensión sin riesgo de pérdida de hechos.

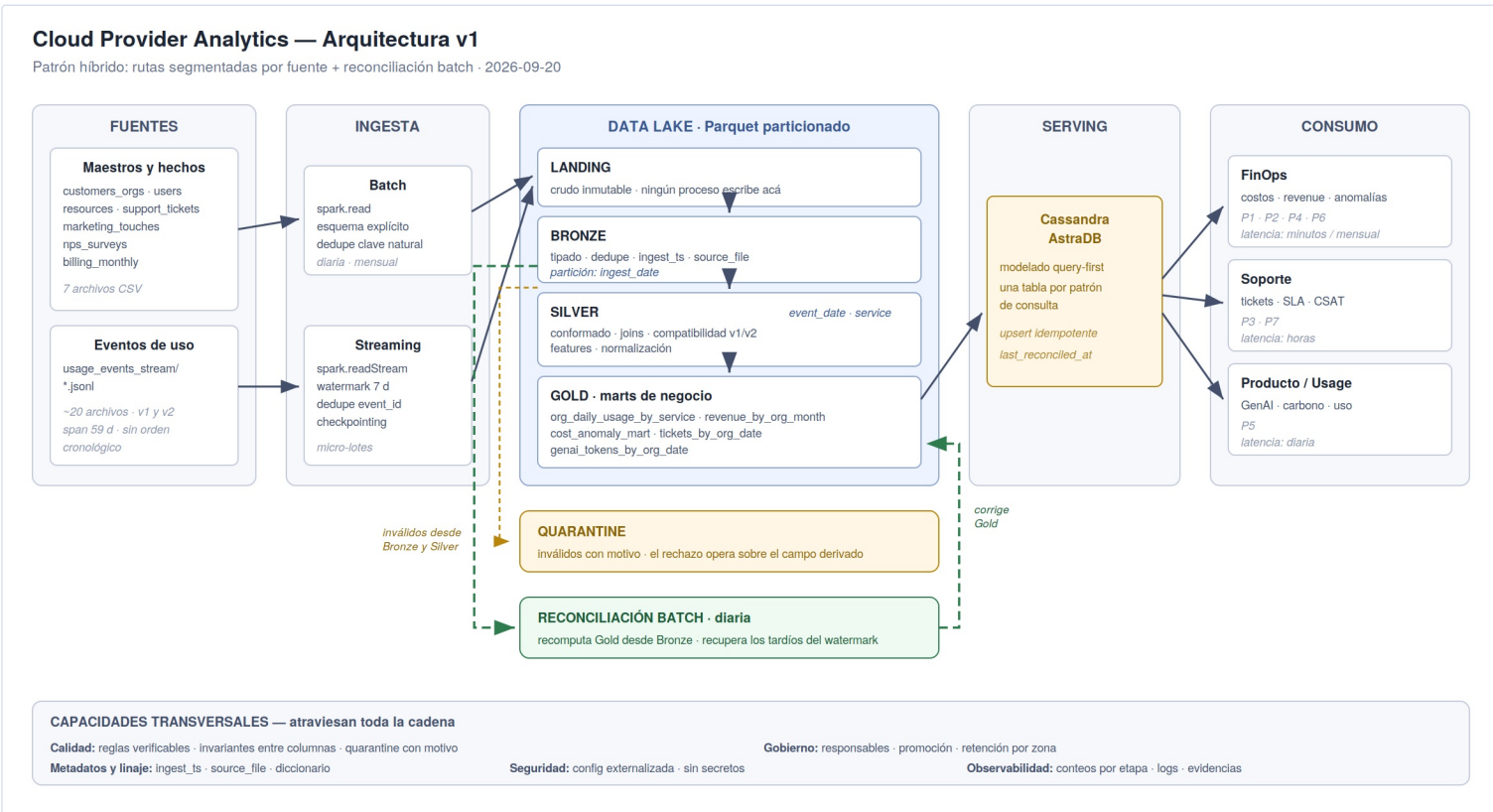
Dos observaciones de cobertura: 20 organizaciones sin encuesta NPS y 2 sin eventos de uso en esta muestra. Los marts deben tolerar esas ausencias en lugar de filtrarlas silenciosamente.

4.5 Síntesis de riesgos de calidad

#	Riesgo	Fuente	Evidencia	Tratamiento previsto
R1	Tipo ambiguo en <code>value</code>	eventos	9 nulos; emisión mixta número/texto	Cast con fallback controlado; el fallido va a quarantine
R2	Costos negativos y outliers	eventos	mín. -19,82; 6 eventos $ z >3$	Flag de anomalía, nunca descarte silencioso
R3	Evolución de esquema v1→v2	eventos	corte 2025-07-18; regla compuesta genai	Promoción a esquema ampliado con nulos explícitos
R4	Llegada tardía estructural	eventos	archivos con span de 59 días	Watermark + dedupe inter-lote con estado
R5	FX ruidoso sobre USD	billing	160/160 filas USD con tasa en [0,855; 1,118] en vez de 1,0; $\pm 15\%$ por factura	Forzar tasa 1,0 para USD y marcar la fila (\$8.8)
R6	Invariantes temporales violadas	users	232/661 con <code>last_login < created_at</code>	Regla entre columnas; flag, no descarte
R7	Inconsistencia lógica de embudo	marketing	96 <code>converted</code> sin <code>clicked</code>	Regla entre columnas
R8	Duplicados de clave natural	tickets, marketing, billing	17 / 61 / 9	Deduplicación en Bronze por clave + <code>ingest_ts</code>
R9	NPS redundante y divergente	orgs vs nps_surveys	dos fuentes, valores distintos	Definir fuente de verdad en Silver
R10	Nulos semánticos vs. defectos	tickets	240 <code>resolved_at</code> , 254 <code>csat</code>	No enviar a quarantine; son estado válido

5. Arquitectura de alto nivel

El diagrama completo está en docs/diagrama arquitectura v1.md , versionado y fechado por separado según exige el artefacto "Diagrama de arquitectura v1".



La solución sigue la cadena de referencia de la consigna:

Fuentes → Ingesta → Data Lake → Procesamiento → Serving → Consumo

con calidad, gobierno, metadatos, seguridad y observabilidad como **capacidades transversales**, no como etapas adicionales. La distinción no es cosmética: un control de calidad ubicado como paso intermedio solo valida lo que pasa por ese punto, mientras que como capacidad transversal se aplica en cada frontera entre zonas.

La cadena se recorre por **dos rutas con latencias distintas**, que es la respuesta directa a la tensión planteada en la §2.1. La justificación de por qué son dos y no una es la §6.

5.2 Componentes por etapa

Etap	Componente	Responsabilidad	Tecnología
Fuentes	7 archivos CSV	Maestros y hechos batch	—
Fuentes	usage_events_stream/*.jsonl	Eventos de uso fragmentados en micro-lotes	—
Ingesta	Lector batch	Leer CSV con esquema explícito y escribir Bronze	spark.read
Ingesta	Lector streaming	Leer el directorio JSONL con watermark, dedupe por event_id y checkpoint	spark.readStream
Data Lake	Landing	Crudo inmutable; ningún proceso escribe acá	Archivos originales
Data Lake	Bronze	Mismo grano que la fuente, tipado explícito, deduplicación, ingest_ts y source_file	Parquet particionado
Data Lake	Silver	Normalización, joins, tratamiento de nulos y outliers, compatibilidad v1/v2, features	Parquet particionado
Data Lake	Gold	Marts por dominio, con grano definido y listos para servir	Parquet particionado
Data Lake	Quarantine	Registros inválidos con motivo de rechazo	Parquet
Procesamiento	Conformación	Normalizar regiones, servicios, unidades y tipos; promover v1 al esquema ampliado	PySpark DataFrames
Procesamiento	Features	daily_cost_usd, requests, cpu_hours, storage_gb_hours, genai_tokens, carbon_kg	PySpark
Procesamiento	Anomalías	Score o flag sobre el costo incremental	PySpark (método en A-04)
Serving	Keyspace y tablas	Modelado query-first, una tabla por patrón de consulta	Cassandra / AstraDB
Consumo	FinOps, Soporte, Producto	Consultas CQL y herramienta de BI externa	—

5.3 Las dos rutas

Ruta streaming — eventos de uso. Es la que sostiene el objetivo O1 (≤ 5 min). El perfilado mostró que los archivos no son cortes cronológicos sino muestras del período completo, con un span de 59 días por archivo, lo que convierte la llegada tardía en condición estructural. La ruta requiere por lo tanto watermark dimensionado contra el desorden observado, deduplicación por event_id sostenida en estado e inter-lote, y checkpointing para garantizar O4.

Ruta batch — maestros, hechos y facturación. Opera a dos cadencias: diaria para maestros, tickets, interacciones y encuestas; mensual para la facturación, que en el dataset tiene exactamente tres valores de month. No hay ningún requisito de negocio que justifique streaming acá: forzarlo agregaría complejidad operativa sin reducir ninguna latencia relevante.

Las dos rutas **convergen en Silver**, no antes. Bronze conserva el grano y el formato de cada fuente; es en Silver donde los eventos se enriquecen contra las dimensiones y donde el modelo pasa a ser común.

5.4 Capacidades transversales

Capacidad	Materialización concreta
Calidad	Reglas verificables en cada frontera entre zonas, incluyendo invariantes entre columnas (§3.5). Quarantine en Parquet con motivo de rechazo. Flags de anomalía en lugar de descarte.
Gobierno	Responsable por dominio, reglas de promoción entre zonas, política de retención diferenciada y criterio de nomenclatura (§8).
Metadatos y linaje	ingest_ts y source_file en Bronze como base de trazabilidad hasta el archivo de origen. Diccionario de datos y correspondencia entre marts y preguntas P1–P7.
Seguridad	Configuración externalizada, ausencia de credenciales en el repositorio, credenciales de AstraDB por variable de entorno, acceso diferenciado por rol de consumo.
Observabilidad	Conteos de filas leídas, promovidas y rechazadas por etapa y por corrida; logs de ejecución; evidencias persistidas en evidence/.

Las cinco están declaradas acá y se especifican en la §8 (gobierno y retención) y en la §9 (calidad y observabilidad en cada flujo).

5.5 Entorno de ejecución y supuestos

Supuesto S-01: el entorno de ejecución es **Google Colab con PySpark en modo local (master=local[*])**, que es lo que la consigna admite explícitamente. El perfilado de la §4 ya se ejecutó sobre PySpark 4.2.0, la misma versión de la Clase 03.

Esto tiene dos consecuencias que el diagrama debe reflejar con honestidad, y que se asumen de forma deliberada:

- **El paralelismo es simulado.** local[*] usa los núcleos de una sola máquina virtual. La API, el plan de ejecución, el particionado y el DAG son los mismos que en un clúster real, pero no hay distribución entre nodos. El diseño de particiones se justifica contra el caso productivo dimensionado en la §3.2, no contra el runtime de la demo.
- **El almacenamiento es efímero.** El sistema de archivos de Colab se pierde al cerrar la sesión. Las zonas del Data Lake se materializan en disco local durante la corrida y el estado persistente real vive en Cassandra/AstraDB, que sí es externo. Los checkpoints de streaming son locales y se reinician con la sesión, lo que obliga a que el pipeline sea reproducible desde cero en lugar de incremental entre sesiones.

En una implementación productiva, Landing, Bronze, Silver y Gold vivirían en almacenamiento de objetos (S3, GCS o ADLS) y el motor correría sobre YARN o Kubernetes. Esa diferencia se documenta como limitación conocida y no se representa en el diagrama v1, porque el diagrama debe mostrar lo que se va a construir.

Si el entorno cambia —una VM con disco persistente, Databricks Community o un clúster propio— lo que se modifica es esta subsección y el bloque de almacenamiento del diagrama; las zonas, los flujos y el modelo de serving no se ven afectados. El diseño se mantiene portable a propósito.

6. Patrón arquitectónico: elección y descarte

6.1 El requisito invariable

La consigna fija un piso, independientemente del patrón que se elija: la solución debe implementar **al menos streaming de eventos y batch de maestros y facturación**, explicando por qué la elección es adecuada. Los tres candidatos admitidos son Lambda, Kappa e híbrido.

Lo que sigue no es una preferencia estética. Los tres patrones se evalúan contra los objetivos de la §2.4 y contra la evidencia del perfilado.

6.2 Criterios de evaluación

Criterio	Origen
C1 · Cumple O1 (frescura ≤ 5 min)	§2.4
C2 · Cumple O2 sin complejidad injustificada	§2.4
C3 · Tolera llegada tardía estructural	§3.3 — archivos con span de 59 días
C4 · Sostiene O4 (reprocesamiento sin duplicados)	§2.4
C5 · Costo de mantenimiento: cantidad de lógica duplicada	Clase 03
C6 · Viable en el entorno declarado (S-01, Colab local[*])	§5.5

6.3 Por qué no Kappa

Kappa trata todas las fuentes como streams y deriva Silver y Gold por re-stream o backfill. Su virtud es tener **una sola base de código**.

Falla en C2 y en C6.

`billing_monthly.csv` tiene exactamente **3 valores distintos de month** en 240 filas: es una fuente que se consolida una vez por mes. Modelarla como stream implica checkpoints, estado y watermarks para procesar un evento mensual. Lo mismo con los 80 registros de `customers_orgs.csv`. No hay ninguna latencia que esa complejidad reduzca.

A eso se suma que Kappa depende de poder **reproducir el log completo** para backfill. En el entorno declarado el almacenamiento es efímero y no hay un bus de eventos persistente tipo Kafka: el re-stream se haría releyendo archivos, que es batch con otro nombre.

Se descarta. Sería la opción correcta si las fuentes llegaran todas por un log persistente y el equipo pagara el costo operativo de una plataforma de streaming real.

6.4 Por qué tampoco Lambda en su forma canónica

Acá hay una distinción que conviene hacer con precisión, porque es donde la mayoría de las soluciones se declara "Lambda" sin serlo.

Lambda canónico procesa **los mismos datos** por dos caminos en paralelo: una capa batch que recomputa la vista completa desde el dato crudo y una capa de velocidad que produce una vista incremental aproximada. La capa de serving **fusiona ambas** al momento de la consulta, y la batch va corrigiendo lo que la de velocidad aproximó.

Lo que este caso necesita no es eso. Acá los eventos van por streaming y los maestros por batch: son **fuentes distintas por caminos distintos**, sin recomputación duplicada, sin dos vistas del mismo dato y sin fusión en la consulta.

Llevarle Lambda a eso sería impreciso. No hay dos implementaciones de la misma lógica, que es justamente el costo que define al patrón (C5) y la razón por la que Kappa existe como reacción.

6.5 Decisión: híbrido segmentado con reconciliación acotada

D-004 · Se adopta un patrón híbrido. Cierra la decisión abierta A-01.

El patrón tiene dos componentes, y el segundo es el que lo hace híbrido de verdad y no simplemente "batch y streaming conviviendo".

Componente 1 — Segmentación por fuente. Cada fuente se enruta según la latencia que su decisión de negocio exige (§2.2):

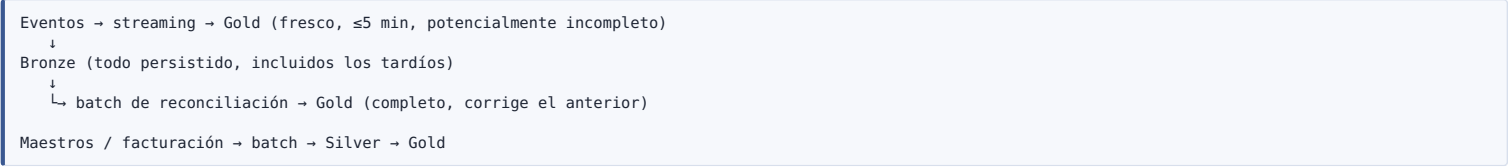
Ruta	Fuentes	Cadencia	Justificación
Streaming	<code>usage_events_stream/*.jsonl</code>	Micro-lotes	O1: detectar anomalías antes del cierre
Batch diario	maestros, tickets, interacciones, encuestas	Diaria	Ninguna decisión requiere menos latencia
Batch mensual	<code>billing_monthly.csv</code>	Mensual	La fuente se consolida una vez por mes

Componente 2 — Reconciliación batch sobre la ruta de eventos. Es el elemento genuinamente Lambda, y está justificado por un hallazgo concreto del perfilado, no por adherir a un patrón.

El problema: cada archivo de eventos cubre **59 días sin orden cronológico** (§3.3). Para cumplir O1 el watermark tiene que ser corto —del orden de días, no de meses—, porque un watermark de 59 días implicaría sostener dos meses de estado y no finalizar ninguna ventana hasta dos meses después. Pero un watermark corto **descarta** los eventos que llegan más tarde que su umbral.

La consecuencia es inevitable: **la ruta de streaming va a producir agregados incompletos**. No por un error de implementación, sino por la naturaleza de la fuente.

La solución es un **job batch periódico que recomputa los marts Gold desde Bronze**, donde los eventos tardíos sí quedaron persistidos aunque la agregación en streaming los haya ignorado. Streaming entrega fresca; batch entrega corrección. Esto es exactamente el problema que Lambda fue diseñado para resolver, aplicado solo donde hace falta.



La recomputación es **idempotente por construcción**: recalcula la partición completa a partir de Bronze y la sobrescribe, con lo que sostiene O4 sin lógica adicional.

6.6 Evaluación contra los criterios

	Kappa	Lambda canónico	Híbrido segmentado
C1 · Frescura ≤ 5 min	✓	✓	✓
C2 · Sin complejidad injustificada	✗ maestros como stream	✗ recomputación duplicada de todo	✓
C3 · Tolerancia llegada tardía	⚠ depende del backfill	✓	✓ vía reconciliación
C4 · Reprocesamiento sin duplicados	✓	✓	✓
C5 · Lógica duplicada	ninguna	alta	acotada a los marts de eventos
C6 · Viable en Colab	✗ sin log persistente	⚠ costoso	✓

6.7 Lo que esta decisión cuesta

Un patrón elegido sin declarar su costo no está justificado, está defendido.

- **Hay lógica duplicada**, aunque acotada: la agregación de `org_daily_usage_by_service` se implementa en streaming y en batch. Se mitiga factorizando la transformación en una función compartida que ambas rutas invocan, en línea con el patrón `extract / transform / validate / load` de la Clase 03.
- **Hay una ventana de inconsistencia temporal**. Entre la corrida de streaming y la de reconciliación, un mismo mart puede devolver valores distintos. Se mitiga exponiendo en el mart la marca de la última reconciliación, para que el consumo sepa qué está mirando.
- **Hay dos rutas que mantener**. Es el costo inherente a no elegir Kappa, y se paga a cambio de no modelar 80 filas mensuales como un stream.

6.8 Cuándo revisaríamos la decisión

- Si apareciera un bus de eventos persistente que permita re-stream real, Kappa pasaría a ser competitivo y eliminaría la duplicación de C5.
- Si el perfilado del dataset completo (~20 archivos) mostrara que los eventos sí llegan ordenados, la reconciliación dejaría de ser necesaria y el patrón se reduciría a segmentación simple. **Esta verificación queda pendiente**: el perfilado se hizo sobre un solo archivo.
- Si se adoptara un formato de lakehouse con transacciones (Delta, Iceberg), los upserts sobre Gold reemplazarían la recomputación completa de la partición.

7. Matriz requisito-componente y mapeo 5V → decisión

7.1 Cómo leer esta sección

La consigna pide mapear requisitos a componentes **incluyendo la relación entre las 5V y las decisiones de arquitectura**. La trazabilidad se tiende en cuatro matrices encadenadas, de lo conceptual a lo verificable:

5V → decisión de arquitectura → componente → evidencia
P1–P7 → mart Gold → tabla de serving → consulta
01–08 → componente → cómo se verifica
Requisitos §4.4 de la consigna → componente → instancia de entrega

Ninguna fila queda sin destino. Lo que no se cubre está declarado en la §7.6.

7.2 Las 5V y la decisión que fuerzan

V	Evidencia	Decisión de arquitectura	Componente	§
Volumen	0,0125 eventos por recurso-hora: muestra diezmada de un caso que escala a miles de millones anuales	Almacenamiento columnar particionado y procesamiento distribuido; retención diferenciada por zona	Parquet particionado en las cuatro zonas	3.2, 8
Velocidad	Dos latencias incompatibles: minutos para FinOps, mensual para facturación	Segmentación de rutas por fuente según la latencia que su decisión exige	Ruta streaming + ruta batch diaria + ruta batch mensual	3.3, 6.5
Velocidad	Archivos con span de 59 días sin orden cronológico	Watermark corto + deduplicación inter-lote con estado + reconciliación batch	Structured Streaming con checkpoint; job de reconciliación desde Bronze	3.3, 6.5
Variedad	CSV y JSONL; <code>schema_version</code> 1 y 2 conviviendo; <code>tags_json</code> anidado	Esquemas explícitos en toda lectura; una capa cuya responsabilidad es conformar	Silver: promoción de v1 al esquema ampliado, parseo de <code>tags_json</code> , normalización	3.4, 8
Veracidad	10 riesgos documentados; 35% de invariantes temporales violadas; FX sobre USD	Reglas con invariantes entre columnas; quarantine con motivo; flags en lugar de descarte	Controles en cada frontera entre zonas; zona Quarantine	3.5, 9
Valor	Anomalías accionables, SLA cuantificado, adopción GenAI, carbono	Marts orientados a pregunta, no a fuente; modelado query-first	Gold + tablas de Cassandra derivadas de P1–P5	3.6, 8

7.3 Preguntas de negocio → marts → serving

Pregunta	Mart Gold	Grano	Fuentes que lo alimentan	Ruta
P1 · Costo y requests diarios por org y servicio	<code>org_daily_usage_by_service</code>	org, fecha, servicio	eventos + <code>customers_orgs</code> + <code>resources</code>	Streaming + reconciliación
P2 · Top-N servicios por costo en 14 días	<code>org_daily_usage_by_service</code>	org, fecha, servicio	ídem P1	Streaming + reconciliación
P3 · Tickets críticos y tasa de SLA por día	<code>tickets_by_org_date</code>	org, fecha, severidad	<code>support_tickets</code> + <code>customers_orgs</code>	Batch diaria
P4 · Revenue mensual neto, normalizado a USD	<code>revenue_by_org_month</code>	org, mes	<code>billing_monthly</code> + <code>customers_orgs</code>	Batch mensual
P5 · Tokens GenAI y costo estimado por día	<code>genai_tokens_by_org_date</code>	org, fecha	eventos v2 con <code>service = genai</code>	Streaming + reconciliación
P6 · Consumos desviados de su histórico	<code>cost_anomaly_mart</code>	org, fecha, servicio	derivado de <code>org_daily_usage_by_service</code>	Batch diaria
P7 · Relación soporte ↔ riesgo de abandono	—	—	<code>support_tickets</code> + <code>nps_surveys</code> + <code>customers_orgs</code>	Deseable, no obligatoria

Nota de modelado que conviene anticipar. P1 y P2 leen el **mismo mart Gold** pero necesitan **tablas de serving distintas** en Cassandra. P1 filtra por rango de fechas, con lo que la clustering key ordena por fecha; P2 pide un Top-N por costo acumulado, que exige ordenar por costo descendente. En un modelo query-first la tabla se diseña desde la consulta, no desde la entidad: una sola tabla no resuelve ambas sin escaneo en el cliente. El diseño concreto de claves de partición y clustering es materia de la §8.

7.4 Objetivos medibles → componente → verificación

Obj.	Componente que lo sostiene	Cómo se verifica	Instancia
O1 · Frescura ≤ 5 min	Ruta streaming con watermark corto	Medición de latencia evento → mart consultable	2. ^a
O2 · Completitud batch	Ingesta batch de los 7 maestros y hechos	Conteos por fuente y por corrida	2. ^a
O3 · Calidad trazable	Reglas en cada frontera + zona Quarantine	Muestras de quarantine con motivo de rechazo	2. ^a
O4 · Sin duplicados al reprocesar	Checkpoint + dedupe por event_id + sobrescritura de partición en la reconciliación	Conteos antes/después de re-ejecutar	2. ^a
O5 · Cobertura analítica	Los cinco marts de la §7.3	Ejecución de P1-P5 sobre Cassandra	Final
O6 · Latencia de consulta < 1 s	Modelado query-first, una tabla por patrón	Cronometrado de las cinco consultas	Final
O7 · Reproducibilidad	Configuración externalizada + README Quickstart	Ejecución desde entorno limpio	Final
O8 · Compatibilidad v1/v2	Promoción en Silver al esquema ampliado con nulos explícitos	Consulta que devuelve eventos de ambas versiones sin ruptura	2. ^a

7.5 Requisitos técnicos obligatorios de la consigna → componente

Trazabilidad contra los doce requisitos de la §4.4 de la consigna, para verificar que ninguno quedó sin asignar.

Requisito	Componente propuesto	§	Entrega
Ingesta batch	Lector con esquema explícito → Bronze Parquet particionado, con ingest_ts y source_file	5.2, 9	2. ^a
Ingesta streaming	readStream sobre el directorio JSONL, esquema explícito, watermark, dedupe por event_id, late data, checkpoint	5.3, 9	2. ^a
Calidad	Reglas verificables por frontera, incluyendo invariantes entre columnas; quarantine en Parquet	3.5, 9	2. ^a
Silver	Normalización de números, fechas, regiones y servicios; joins con dimensiones; nulos y outliers; compatibilidad v1/v2	5.2, 8	2. ^a
Features	daily_cost_usd, requests, cpu_hours, storage_gb_hours, genai_tokens, carbon_kg	5.2	2. ^a
Anomalías	Flag o score sobre el costo incremental; método pendiente en A-04	3.5, 11	2. ^a
Gold	Cinco marts con grano declarado en la §7.3	7.3, 8	2. ^a y final
Serving	Keyspace y tablas query-first; carga desde Spark	5.2, 8	2. ^a
Idempotencia	Checkpoints, claves naturales, upserts y sobrescritura de partición en la reconciliación	6.5, 9	2. ^a
Performance	Particionado justificado, control de small files, coalesce / repartition	8	2. ^a y final
Gobierno	Responsables por dominio, metadatos, linaje, seguridad y observabilidad como capacidades transversales	5.4, 8	2. ^a y final
Documentación	Diagrama versionado, diccionario de datos, DECISIONS.md, Quickstart y evidencias	13, repo	las tres

7.6 Qué no queda cubierto, y por qué

Declarar la cobertura sin declarar los huecos la vuelve poco creíble.

Hueco	Motivo	Tratamiento
P7 (soporte ↔ churn) no tiene mart asignado	20 de 80 organizaciones sin encuesta NPS y 25,4% de tickets sin CSAT. La señal es débil para un modelo supervisado	Declarada deseable. Si no alcanza, la consigna admite "una alternativa equivalente acordada"
Performance real no verificable	El entorno declarado es local[*] : el paralelismo es simulado	El particionado se justifica contra el caso dimensionado en la §3.2, no contra el runtime de la demo
Retención no ejercitable	El dataset cubre 4 meses; una política de retención a 12 o 24 meses no se puede demostrar	Se especifica en la §8 y se declara como no verificable en esta instancia
Reconciliación pendiente de validar	El perfilado se hizo sobre 1 de ~20 archivos (A-07)	Se verifica al recibir el dataset completo

8. Diseño del Data Lake

8.1 Principios

- 1. **Landing es inmutable.** Ningún proceso escribe en landing/. Es la única copia del dato tal como llegó y la base de cualquier reprocesamiento.
- 2. **Cada zona tiene una responsabilidad y una sola.** Si una transformación puede hacerse en dos zonas, va en la de mayor grano que la haga posible.
- 3. **Toda promoción es idempotente.** Volver a ejecutar cualquier paso produce el mismo resultado, sin duplicados ni residuos.
- 4. **El particionado se justifica contra el caso dimensionado en la §3.2,** no contra el runtime de la demo (§5.5).
- 5. **Ningún registro se descarta en silencio.** Lo inválido va a quarantine con motivo; lo sospechoso se marca con flag y sigue.

8.2 Zonas

Zona	Responsabilidad	Formato	Grano	Escribe
Landing	Archivos originales, sin intervención	CSV / JSONL	el de la fuente	nadie
Bronze	Tipado explícito, deduplicación por clave natural, columnas técnicas	Parquet	el de la fuente	ingesta batch y streaming
Silver	Normalización, joins con dimensiones, nulos y outliers, compatibilidad v1/v2, features	Parquet	conformado por entidad	procesamiento
Gold	Marts por dominio, listos para servir	Parquet	el declarado en la §7.3	procesamiento
Quarantine	Registros rechazados con motivo y origen	Parquet	el del registro rechazado	controles de calidad

Quarantine es una zona y no una columna: si el inválido queda como flag dentro de la tabla, toda consulta posterior tiene que acordarse de filtrarlo.

8.3 Particionado

D-005 · Estrategia de particionado. Cierra A-02.

El problema que hay que resolver primero. El perfilado mostró que los archivos de eventos cubren 59 días sin orden cronológico (§3.3). Si Bronze se

particionara por **fecha del evento**, cada micro-lote escribiría en casi todas las particiones del histórico, generando decenas de archivos diminutos por corrida. Es el problema de small files en su forma más aguda.

La decisión: Bronze particiona por fecha de ingesta, Silver y Gold por fecha del evento.

Zona	Clave de partición	Por qué
Bronze eventos	ingest_date	Cada micro-lote escribe en una sola partición. Bronze responde "qué llegó y cuándo"
Bronze maestros	sin particionar (volumen bajo) o ingest_date	80 a 1.500 filas: particionar sería contraproducente
Silver eventos	event_date	Silver responde "qué pasó y cuándo": es la semántica analítica
Gold marts diarios	usage_date	Alineado con el grano del mart y con el filtro de P1 y P3
Gold revenue	month	Grano mensual, 3 particiones en el dataset
Quarantine	ingest_date + regla_violada	Permite auditar una regla sin escanear todo

El cambio de eje entre Bronze y Silver **no es un capricho: es lo que separa auditoría de análisis**. Y tiene un beneficio operativo directo sobre el patrón de la §6: el job de reconciliación puede leer solo las particiones de Bronze con ingest_date reciente para saber **qué fechas de evento fueron tocadas por datos tardíos**, y recomputar únicamente esas particiones de Gold en lugar de todo el histórico.

Segundo nivel de partición. En Silver y Gold de eventos se agrega service como segunda clave: tiene cardinalidad 6, estable y conocida, y P1 y P2 filtran o agrupan por él. No se usa org_id como clave de partición: con 80 valores hoy y miles en el caso dimensionado, produciría particiones diminutas y un directorio inmanejable.

Dimensionamiento. En el escenario mediano de la §3.2 (3,6 millones de eventos diarios, ~300 bytes cada uno) una partición diaria ronda el gigabyte crudo, y dividida por servicio quedan bloques del orden de las decenas de megabytes comprimidos: el rango razonable para Parquet.

Honestidad sobre la muestra. Con el dataset provisto, 7.200 eventos en 60 días dan ~120 eventos por día: particionar por fecha y servicio produce particiones de unas pocas decenas de filas. **A escala de la muestra el esquema está sobre-particionado**, y se compensa con coalesce(1) antes de escribir para no generar cientos de archivos triviales. El esquema se diseña para el caso, no para la demo; la evidencia de tamaños y rutas se presenta en la segunda entrega.

8.4 Convenciones de nomenclatura

```
data/
  landing/<fuente>.csv | usage_events_stream/*.jsonl
  bronze/<entidad>/ingest_date=YYYY-MM-DD/
  silver/<entidad>/event_date=YYYY-MM-DD/service=<servicio>/
  gold/<mart>/usage_date=YYYY-MM-DD/
  quarantine/<entidad>/ingest_date=YYYY-MM-DD/regla=<regla>/
  _checkpoints/<job>/
```

- Entidades y marts en snake_case, en singular para dimensiones y en plural para hechos.
- Los marts conservan los nombres de referencia de la consigna (org_daily_usage_by_service, revenue_by_org_month, cost_anomaly_mart, tickets_by_org_date, genai_tokens_by_org_date).
- Las claves de partición usan el estilo clave=valor de Hive, que es el que Spark interpreta para poda de particiones.
- Los checkpoints viven fuera de las zonas de datos: no son dato.

8.5 Retención

Zona	Retención	Fundamento
Landing	Indefinida	Es la única copia del crudo; sin ella no hay reprocesamiento posible
Bronze	24 meses	Permite backfill completo y auditoría; es el insumo de la reconciliación
Silver	24 meses	Alineado con Bronze: se puede regenerar desde ahí
Gold	36 meses	Horizonte de análisis interanual; es la capa más compacta
Quarantine	6 meses	Sirve para diagnóstico, no para histórico
Checkpoints	Vida del job	Se descartan al reiniciar el pipeline desde cero

Limitación declarada. El dataset cubre 4 meses, con lo que ninguna política de retención es ejercitable en esta instancia. Se especifica porque la consigna pide la definición, no su ejecución.

8.6 Metadatos y linaje

Columnas técnicas obligatorias en Bronze, que son la base de toda la trazabilidad:

Columna	Contenido	Para qué
ingest_ts	Timestamp de la corrida de ingesta	Linaje temporal; acota la reconciliación
source_file	Ruta del archivo de origen	Trazabilidad hasta el archivo exacto
schema_version	Versión del esquema de origen (eventos)	Compatibilidad v1/v2

En Silver se suman quality_flags (array de reglas marcadas pero no bloqueantes) y is_anomaly con su score. En Gold se agrega last_reconciled_at, que permite al consumo distinguir un valor provisorio del streaming de uno consolidado por la reconciliación (§6.7).

El diccionario de datos completo —campo, tipo, origen, regla y responsable— se entrega con la implementación en la segunda instancia.

8.7 Reglas de promoción entre zonas

Frontera	Condición para promover	Qué va a Quarantine
Landing → Bronze	Parseable con el esquema declarado; clave natural presente	Registro no parseable o sin clave natural
Bronze → Silver	Pasa las reglas de calidad bloqueantes	value no casteable; unit nulo con value presente; invariantes temporales violadas
Silver → Gold	Grano y claves completos; dimensiones resueltas	Nada: en Gold ya no hay inválidos, solo flags

La distinción que gobierna estas reglas es la de nulo semántico frente a defecto (§3.5). Los 240 resolved_at nulos son tickets abiertos y los 254 csat nulos son encuestas no respondidas: son estado válido y **promueven**. Los 17 unit nulos con value presente son defectos y **no promueven**. El criterio: un nulo es semántico si existe una lectura de negocio que lo explique, y es defecto si viola una regla declarada.

Los costos negativos **promueven con flag**, nunca se rechazan: un valor de −19,82 puede ser una nota de crédito legítima.

8.8 Decisiones de conformación

D-006 · Fuente de verdad del NPS. Cierra A-03.

El NPS aparece en dos lugares con valores que no coinciden: `customers_orgs.nps_score` (80 organizaciones, 11 nulos) y `nps_surveys.csv` (92 encuestas sobre 60 organizaciones, 32 de ellas con más de una).

Decisión. `nps_surveys` es la fuente de verdad, porque es el hecho con fecha; `customers_orgs.nps_score` es una foto desnormalizada de antigüedad desconocida. En Silver la dimensión conserva **ambos valores con nombres distintos**, `nps_score_crm` y `nps_score_last_survey`, más `nps_survey_date`. No se sobrescribe uno con otro: se expone la discrepancia y Gold elige.

El caso borde que obliga a esto: 20 de las 80 organizaciones nunca fueron encuestadas. Si se eliminara la columna del CRM, esas 20 quedarían sin NPS. La regla de consumo es usar la última encuesta y caer al valor del CRM cuando no exista, marcando el origen.

D-007 · Normalización de moneda a USD. Cierra A-06.

El hallazgo. Las tasas de ARS (mediana 0,00150) y EUR (mediana 1,10467) son plausibles. Las 160 filas en USD tienen tasas entre 0,85463 y 1,11791 en lugar de 1,0. Es ruido alrededor de 1: convertir dólares a dólares tiene tasa 1 por definición.

Decisión. `amount_usd` = `subtotal_neto` cuando `currency` = 'USD', y `subtotal_neto` × `exchange_rate_to_usd` en cualquier otro caso. La fila se marca en `quality_flags` como `fx_usd_ruidoso` para que la corrección sea auditable, y no se envía a `quarantine` porque el dato de negocio es recuperable.

Por qué no confiar en el campo tal como viene. En el agregado el ruido se compensa y la diferencia de `revenue` total es del 0,1%, así que una validación global no lo detectaría. Pero por factura la distorsión llega a −15% / +12%, y los marts de FinOps reportan por organización y por mes. Un error que se cancela en el total sigue siendo un error en cada fila.

Tercera decisión, menor pero necesaria: `credits` nulo en 137 de 240 filas (57%) significa ausencia de crédito y se resuelve como 0 antes de restar. Si se propagara como nulo, el `revenue` neto de más de la mitad del dataset sería nulo.

8.9 Del Gold al serving

Las tablas de Cassandra se derivan de los marts, no los reemplazan. El principio es query-first: **la tabla se diseña desde la consulta**, con lo que un mart puede originar más de una tabla.

Tabla de serving	Clave de partición	Clustering	Responde
<code>usage_by_org_date_service</code>	<code>(org_id, usage_date)</code>	<code>service</code>	P1
<code>usage_by_org_service_cost</code>	<code>(org_id, periodo)</code>	<code>total_cost DESC, service</code>	P2
<code>tickets_by_org_date</code>	<code>(org_id)</code>	<code>date DESC, severity</code>	P3
<code>revenue_by_org_month</code>	<code>(org_id)</code>	<code>month DESC</code>	P4
<code>genai_by_org_date</code>	<code>(org_id)</code>	<code>date DESC</code>	P5
<code>cost_anomalies_by_org_date</code>	<code>(org_id, date)</code>	<code>score DESC, service</code>	P6

Las dos primeras salen del **mismo mart** `org_daily_usage_by_service` y existen por separado porque en Cassandra el orden lo fija la clustering key, no la consulta (§7.3): P1 filtra por rango de fechas y P2 corta un Top-N por costo. La desnormalización no es un defecto del modelo, es el modelo.

Gold en Parquet se conserva igualmente como capa de exploración ad-hoc: Cassandra responde rápido las preguntas conocidas, pero no las que todavía no se modelaron.

9. Flujos de datos batch y streaming

9.1 Los cuatro flujos

Flujo	Fuentes	Cadencia	Salida	Herramienta
A · Batch diario	maestros, tickets, interacciones, encuestas	Diaria	Bronze → Silver → Gold Soporte	<code>spark.read</code> + DataFrame API
B · Batch mensual	<code>billing_monthly.csv</code>	Mensual	Gold <code>revenue_by_org_month</code>	<code>spark.read</code> + DataFrame API
C · Streaming	<code>usage_events_stream/*.jsonl</code>	Micro-lotes	Bronze → Silver → Gold FinOps/Producto	<code>spark.readStream</code> + <code>foreachBatch</code>
D · Reconciliación	Bronze de eventos	Diaria	Gold FinOps/Producto (corregido)	<code>spark.read</code> + DataFrame API

D no es un flujo accesorio: es lo que garantiza que los marts sean correctos, por las razones que la §9.5 cuantifica.

9.2 Flujo A — batch diario

```
landing/<fuente>.csv
| spark.read.schema(esquema_explicito).csv(...)      ← nunca inferSchema
↓
+ ingest_ts = current_timestamp()
+ source_file = input_file_name()
| dropDuplicates([clave_natural]) ordenado por ingest_ts
↓
bronze/<entidad>/                                     ← Parquet, mode("overwrite")
| reglas de calidad bloqueantes →inválidos→ quarantine/
| normalización de regiones, servicios, fechas
| join con dimensión de organizaciones (left)
↓
silver/<entidad>/
| agregación al grano del mart
↓
gold/tickets_by_org_date/usage_date=.../
| foreachPartition → escritura en Cassandra (upsert)
↓
Cassandra: tickets_by_org_date
```

La deduplicación en Bronze es necesaria por evidencia, no por precaución: el perfilado encontró 17 `ticket_id`, 61 `touch_id` y 9 `invoice_id` repetidos. El criterio es conservar el registro de `ingest_ts` más reciente.

Los joins contra la dimensión son `left` y nunca `inner`: 20 de 80 organizaciones no tienen encuesta NPS, y un `inner` las eliminaría de los marts en silencio.

9.3 Flujo B — batch mensual

Idéntico al anterior en su estructura, con dos transformaciones propias que implementan D-007:

```
neto      = subtotal - coalesce(credits, 0) + taxes      # credits nulo = 0
fx_final  = when(currency == "USD", lit(1.0)).otherwise(exchange_rate_to_usd)
amount_usd = neto * fx_final
```

La fila cuyo `exchange_rate_to_usd` venía distinto de 1,0 con `currency = 'USD'` se marca en `quality_flags` como `fx_usd_ruidoso`. No va a quarantine: el dato de negocio es recuperable y la corrección queda auditable.

9.4 Flujo C — streaming de eventos

```
eventos = (spark.readStream
  .schema(ESQUEMA_EVENTOS)           # explícito; value como String
  .option("maxFilesPerTrigger", 1)    # un archivo = un micro-lote
  .json("landing/usage_events_stream/")
  .withColumn("ingest_ts", F.current_timestamp())
  .withColumn("source_file", F.input_file_name())
  .withColumn("event_ts", F.to_timestamp("timestamp"))
  .withWatermark("event_ts", "7 days")
  .dropDuplicates(["event_id", "event_ts"]) # dedupe inter-lote con estado
)

(eventos.writeStream
  .option("checkpointLocation", "_checkpoints/eventos_bronze")
  .partitionBy("ingest_date")      # D-005: una partición por micro-lote
  .foreachBatch(procesar_lote)
  .start())
```

Cuatro puntos que la consigna exige explícitamente y dónde se resuelven:

Requisito	Resolución
Esquema explícito	ESQUEMA_EVENTOS con 13 campos; value como String por tipo ambiguo
Watermark	withWatermark("event_ts", "7 days") — dimensionado en la §9.5
Deduplicación por event_id	dropDuplicates con watermark , lo que la hace inter-lote y no intra-lote
Late data	Los eventos fuera del watermark se descartan de la agregación pero persisten en Bronze ; los recupera el flujo D
Checkpointing	checkpointLocation, que sostiene O4

`maxFilesPerTrigger = 1` hace que cada archivo sea un micro-lote, que es exactamente la simulación que la fragmentación intencional busca.

El uso de `foreachBatch` permite aplicar sobre cada micro-lote la **misma función de transformación** que usa el flujo D, cumpliendo la mitigación de lógica duplicada declarada en la §6.7.

9.5 Dimensionamiento del watermark

D-008 · Watermark de 7 días sobre event_ts. Cierra A-05.

Structured Streaming descarta de la agregación las filas cuyo event-time es anterior a `max(event_time) - watermark`. Como **cada archivo cubre el rango completo del dataset**, ya el primer micro-lote lleva el máximo a 2025-08-31. Desde el segundo en adelante, todo lo anterior al umbral se descarta.

Cuantificado sobre el archivo perfilado:

Watermark	Eventos vigentes	% procesado	% descartado
1 día	9	2,5%	97,5%
2 días	19	5,3%	94,7%
7 días	32	8,9%	91,1%
14 días	74	20,6%	79,4%
30 días	187	51,9%	48,1%
60 días	360	100%	0%

El resultado es contundente: incluso con un watermark de 30 días se descartaría casi la mitad de los eventos. No es un defecto de la implementación sino la consecuencia directa de que la fragmentación sea aleatoria y no temporal.

Por qué 7 días y no 60. El watermark se dimensiona contra el caso productivo, no contra la muestra (principio 4 de la §8.1). El estado de deduplicación crece con la ventana: en el escenario mediano de la §3.2 (3,6 millones de eventos diarios) una ventana de 7 días implica sostener unos 25 millones de claves, del orden de cientos de megabytes de estado. Una ventana de 60 días implicaría más de 200 millones de claves, inviable en un solo nodo.

Qué significa aceptar esto. La ruta de streaming **no produce marts completos, y no se pretende que lo haga**. Su responsabilidad es la frescura sobre los eventos recientes, que es lo que O1 pide. La completitud la aporta el flujo D.

Esta es la validación empírica más fuerte del patrón elegido en la §6: la reconciliación no es una precaución teórica, es lo único que hace correctos a los marts con estos datos.

Sujeto a A-07. Si el perfilado de los ~20 archivos mostrara que los eventos sí llegan ordenados, el porcentaje descartado caería drásticamente y el watermark podría reducirse.

9.6 Flujo D — reconciliación

```
bronze/eventos/ingest_date=<recientes>
| 1. identificar las event_date tocadas por los últimos lotes
▼
bronze/eventos/ (todas las particiones de ingesta)
| 2. filtrar por esas event_date → incluye los tardíos descartados por C
| 3. misma función de transformación que usa foreachBatch en C
▼
gold/<mart>/usage_date=<tocadas>/ ← mode("overwrite") sobre esas particiones
+ last_reconciled_at = current_timestamp()
▼
Cassandra: upsert
```

Es idempotente por construcción. Recalcula la partición completa desde Bronze y la sobrescribe: no necesita lógica de merge ni control de duplicados. Sostiene O4 sin trabajo adicional.

El paso 1 es lo que habilita el eje de partición elegido en D-005: sin `ingest_date` en Bronze habría que recomputar todo el histórico en cada corrida.

9.7 Reglas de calidad por frontera

Frontera	Regla	Acción	Evidencia que la motiva
Landing → Bronze	<code>event_id</code> no nulo	Quarantine	Clave natural de deduplicación
Landing → Bronze	Registro parseable con el esquema	Quarantine	JSONL con esquema evolutivo
Landing → Bronze	Clave natural única por entidad	Deduplicar	17 / 61 / 9 duplicados hallados
Bronze → Silver	<code>value</code> casteable a double	Quarantine de la métrica , el costo promueve (D-009)	Tipo ambiguo; 9 nulos en la muestra
Bronze → Silver	<code>unit</code> no nulo cuando existe <code>value</code>	Quarantine de la métrica , el costo promueve (D-009)	17 casos hallados
Bronze → Silver	<code>cost_usd_increment</code> >= -0.01	Flag , no rechazo	Puede ser nota de crédito; mínimo −19,82
Bronze → Silver	<code>last_login</code> >= <code>created_at</code>	Flag	232 de 661 usuarios (35,1%)
Bronze → Silver	<code>converted</code> = <code>true</code> ⇒ <code>clicked</code> = <code>true</code>	Flag	96 de 1.500 interacciones
Bronze → Silver	<code>nps_score</code> ∈ [−100, 100]	Flag	1 violación (valor 101)
Bronze → Silver	<code>genai_tokens</code> presente ⇔ <code>v2</code> ∧ <code>service</code> = 'genai'	Flag	24/24 cumplen; la regla ingenua daría 251 falsos positivos
Silver → Gold	Grano completo y dimensiones resueltas	Bloqueante	—

Las tres primeras de Bronze → Silver son las que la consigna exige como mínimo para la segunda entrega. Las cuatro siguientes son **invariantes entre columnas**, que ningún control de completitud detecta (§3.5).

Quarantine frente a flag. Se rechaza lo que impide calcular; se marca lo que resulta sospechoso pero es calculable. Un `value` no casteable impide calcular una feature; un costo negativo se calcula perfectamente y puede ser legítimo.

El rechazo opera sobre el campo derivado, no sobre el registro entero (D-009). Un evento con `value` nulo tiene la métrica inutilizable, pero su `cost_usd_increment` es perfectamente válido. Descartar el registro completo perdería ese costo: en el archivo perfilado, los 9 eventos con `value` nulo y los 17 con `unit` nulo arrastran **el 4,6% del costo total**. El registro promueve con la métrica en nulo y marcada, y una copia va a quarantine para auditoría.

9.8 Idempotencia

Flujo	Mecanismo
A y B	<code>mode("overwrite")</code> sobre la partición + deduplicación por clave natural
C	Checkpoint + <code>dropDuplicates</code> con watermark sobre <code>event_id</code>
D	Recomputación completa de la partición desde Bronze
Serving	Upsert por clave primaria en Cassandra, idempotente por definición

La verificación de O4 es un conteo antes y después de re-ejecutar cada flujo, con diferencia cero.

9.9 Observabilidad

Cada corrida registra: filas leídas, promovidas, deduplicadas y enviadas a quarantine por regla; duración por etapa; rutas y tamaños de salida; y para el flujo C, además, offsets procesados y filas descartadas por watermark.

Esa última métrica no es un detalle: es la que permite detectar que el supuesto de la §9.5 cambió, y es la señal que dispararía la revisión de D-008.

10. Flujo batch de referencia en lógica MapReduce

10.1 Qué job se elige y por qué

Se expresa `org_daily_usage_by_service`, el mart FinOps que la consigna exige como mínimo en la segunda entrega. Es la elección correcta por tres razones: responde P1 y P2, es el único mart que se calcula por las dos rutas (streaming y reconciliación), y su agregación es la que más se beneficia de un combiner.

El objetivo de esta sección no es proponer implementar MapReduce —la solución usa Spark— sino **mostrar que el procesamiento batch del caso se entiende al nivel del modelo de programación**, y dónde Spark se aparta de él.

Entrada: eventos de uso en Bronze. **Salida:** una fila por (`org_id`, `usage_date`, `service`) con costo, requests, cpu_hours, storage_gb_hours, tokens GenAI y carbono.

10.2 Mapper

```
map(clave_entrada, linea_json):
    e = parsear(linea_json)

    # --- compatibilidad v1/v2 (§3.4) ---
    carbon = e.carbon_kg    si e.schema_version == 2    si no NULL
    tokens = e.genai_tokens si (e.schema_version == 2 y e.service == "genai") si no NULL

    # --- casteo con fallback controlado (§3.4) ---
    valor = intentar_double(e.value)    # llega como número o como texto

    # --- calidad a nivel de campo derivado (D-009) ---
    metrica_valida = (valor != NULL) y (e.unit != NULL)
    si no metrica_valida:
        emitir_a_quarantine(e, motivo)
        incrementar_contador("metricas_invalidas")
        valor = NULL    # el costo sigue promoviendo

    # --- proyección al grano del mart ---
    clave = (e.org_id, fecha(e.timestamp), e.service)
    valores = {
        costo:         e.cost_usd_increment,    # siempre, aun si la métrica falló
        requests:      valor si e.metric == "requests"    si no 0,
        cpu_hours:     valor si e.metric == "cpu_hours"    si no 0,
        storage_gb_hours: valor si e.metric == "storage_gb_hours" si no 0,
        tokens:        tokens si tokens != NULL si no 0,
        carbono:       carbon si carbon != NULL si no 0,
        n_eventos:     1
    }
    emitir(clave, valores)
```

Dos decisiones del mapper vale la pena señalar. La **proyección de la métrica a columnas** convierte el formato largo de la fuente (`metric / value`) en el ancho que el mart necesita, y lo hace en el map para que el shuffle mueva menos datos. Y el **costo se emite siempre**, incluso cuando la métrica es inválida: es la aplicación de D-009.

10.3 Combiner

```
combine(clave, lista_de_valores):    # mismo código que el reducer
    emitir(clave, suma_campo_a_campo(lista_de_valores))
```

El combiner es válido **porque todas las agregaciones son sumas**, que son asociativas y conmutativas. Es la condición que lo habilita, y no es un detalle: si el mart necesitara un promedio, el combiner no podría emitir `avg` —habría que emitir `(suma, conteo)` y dividir recién en el reducer—, porque el promedio de promedios no es el promedio.

Por eso `n_eventos` viaja como contador desde el mapper: deja el mart preparado para derivar promedios sin romper la asociatividad.

Su efecto es reducir el volumen del shuffle: con 6 servicios y ~80 organizaciones, muchos eventos de un mismo split colapsan en una sola tupla antes de cruzar la red.

10.4 Partitioner y shuffle/sort

```
particion(clave) = hash(clave.org_id) mod n_reducers
```

Se particiona por `org_id` y no por la clave completa para que **todas las fechas y servicios de una organización caigan en el mismo reducer**. Eso habilita, en un job encadenado, calcular estadísticas por organización sin un segundo shuffle.

El framework ordena por clave y entrega a cada reducer los valores agrupados. Es la etapa cara: implica serialización, escritura a disco y transferencia por red.

Riesgo de skew: si una organización concentrara un volumen desproporcionado de eventos, su reducer se volvería el cuello de botella. En el dataset actual la distribución es pareja (78 organizaciones con eventos), pero a escala productiva las cuentas enterprise podrían desbalancear. La mitigación clásica es agregar un salt a la clave y hacer un segundo pase de agregación.

10.5 Reducer

```
reduce(clave, valores):
    acc = suma_campo_a_campo(valores)
    emitir(clave, {
        org_id, usage_date, service,
        daily_cost_usd:  acc.costo,
        requests:       acc.requests,
        cpu_hours:      acc.cpu_hours,
        storage_gb_hours: acc.storage_gb_hours,
        genai_tokens:   acc.tokens,
        carbon_kg:      acc.carbono,
        event_count:    acc.n_eventos
    })
```

10.6 Trazado con datos reales

Dos claves del archivo perfilado, elegidas porque cada una dispara una regla de calidad distinta.

Entrada (Bronze)

event_id	org	fecha	service	metric	value	unit	cost
evt_gh4g7q9rxnqk	org_cvs4f8cg	2025-08-23	networking	requests	null	count	1,0336
evt_5lwe2oj64bnv	org_cvs4f8cg	2025-08-23	networking	requests	135,0	count	1,3666
evt_ve7w95mgnxs6	org_afeyuhz1	2025-08-11	compute	cpu_hours	3,1704	null	0,2459
evt_srmsqkb6qjpu	org_afeyuhz1	2025-08-11	compute	requests	121,0	count	7,0545

Salida del map

```
(org_cvs4f8cg, 2025-08-23, networking) → {costo:1.0336, requests:0, n:1}  ▲ métrica a quarantine (value nulo)
(org_cvs4f8cg, 2025-08-23, networking) → {costo:1.3666, requests:135, n:1}
(org_afeyuhz1, 2025-08-11, compute)    → {costo:0.2459, cpu_hours:0, n:1}  ▲ métrica a quarantine (unit nulo)
(org_afeyuhz1, 2025-08-11, compute)    → {costo:7.0545, requests:121, n:1}
```

Salida del combiner (colapsa dentro de cada split)

```
(org_cvs4f8cg, 2025-08-23, networking) → {costo:2.4002, requests:135, n:2}
(org_afeyuhz1, 2025-08-11, compute)    → {costo:7.3004, requests:121, cpu_hours:0, n:2}
```

Salida del reducer

org_id	usage_date	service	daily_cost_usd	requests	event_count
org_cvs4f8cg	2025-08-23	networking	2,4002	135	2
org_afeyuhz1	2025-08-11	compute	7,3004	121	2

Lo que el trazado demuestra. Los cuatro eventos tenían defectos en dos de ellos, y aun así **el costo de los cuatro llegó al mart**. Si el registro defectuoso se hubiera descartado entero, el costo del primer par sería 1,3666 en lugar de 2,4002: un 43% menos en esa celda. Extrapolado al archivo completo, la pérdida sería del 4,6% del costo total. Eso es D-009 en acción.

10.7 El mismo job en Spark

```
gold = (bronze_eventos
.withColumn("valor", F.col("value").cast("double"))
.withColumn("metrica_valida", F.col("valor").isNotNull() & F.col("unit").isNotNull())
.withColumn("usage_date", F.to_date("event ts"))
.groupBy("org_id", "usage_date", "service")
.agg(
  F.sum("cost_usd_increment").alias("daily_cost_usd"),
  F.sum(F.when(F.col("metric") == "requests", F.col("valor")).otherwise(0)).alias("requests"),
  F.sum(F.when(F.col("metric") == "cpu_hours", F.col("valor")).otherwise(0)).alias("cpu_hours"),
  F.sum(F.when(F.col("metric") == "storage_gb_hours", F.col("valor")).otherwise(0)).alias("storage_gb_hours"),
  F.sum(F.coalesce("genai_tokens", F.lit(0))).alias("genai_tokens"),
  F.sum(F.coalesce("carbon_kg", F.lit(0))).alias("carbon_kg"),
  F.count("*").alias("event_count"),
))
```

10.8 Correspondencia y diferencias

MapReduce	Spark	Observación
map	withColumn + select	Transformaciones narrow, sin shuffle
combine	Agregación parcial previa al Exchange	Spark lo aplica automáticamente ; no hay que escribirlo
partition por hash	Exchange hashpartitioning	Visible en <code>explain(mode="formatted")</code>
shuffle / sort	Exchange	La etapa cara en ambos modelos
reduce	groupBy().agg()	—
Contadores del job	Acumuladores / métricas de corrida	Alimentan la observabilidad de la §9.9

Dónde la correspondencia se rompe, y por qué importa. Las diferencias son exactamente las que señala la Clase 03:

- **MapReduce escribe a HDFS con replicación entre jobs.** Spark encadena transformaciones en un DAG y mantiene los resultados en memoria.
- **MapReduce inicia una JVM por tarea.** Spark reutiliza las JVM de los ejecutores.
- **En MapReduce el combiner se escribe a mano** y es responsabilidad del programador verificar que la operación sea asociativa. Catalyst lo decide solo.
- **MapReduce ejecuta cada paso al invocarlo.** Spark difiere hasta la acción, lo que le permite podar columnas y empujar filtros antes de leer.

10.9 Dónde la diferencia se vuelve decisiva: el mart de anomalías

El contraste no es académico. El mart `cost_anomaly_mart` (P6) necesita **tres pases** sobre los datos:

1. Agregar el costo diario por organización y servicio — el job de arriba.
2. Calcular la estadística de referencia (mediana, MAD o percentiles) por organización y servicio sobre la serie histórica.
3. Marcar cada día contra esa estadística.

En MapReduce son tres jobs encadenados, y entre cada uno el resultado intermedio **se escribe a HDFS con replicación triple y se vuelve a leer**. El paso 1 produce un volumen pequeño, pero igual paga el ciclo completo de escritura, replicación y lectura, más el arranque de JVM de cada tarea.

En Spark los tres pasos son un solo DAG: el resultado del paso 1 se cachea en memoria y se reutiliza en los pasos 2 y 3. El particionado por `org_id` elegido en la §10.4 hace que el paso 2 no necesite un shuffle adicional, porque los datos de cada organización ya están colocados.

Esto es lo que la Clase 03 resume como la ventaja del cacheo en memoria y los grafos de linaje, y es la razón concreta por la que la solución usa Spark y no MapReduce: **no es que MapReduce no pueda resolver el caso, es que lo resuelve pagando I/O innecesario en cada paso de una cadena que acá tiene tres eslabones.**

11. Supuestos, riesgos, mitigaciones y decisiones abiertas

11.1 Supuestos

Cada supuesto se declara con su impacto si resultara falso, porque un supuesto sin consecuencia declarada no es un supuesto: es una afirmación disfrazada.

#	Supuesto	Si fuera falso
S-01	El entorno de ejecución es Google Colab con PySpark en modo local (\$5.5)	Cambia la \$5.5 y el bloque de almacenamiento del diagrama; zonas, flujos y serving no se ven afectados
S-02	El dataset es una réplica reducida de una carga productiva, no la carga misma (\$3.1)	Se cae la justificación de Big Data y el proyecto sería un ejercicio de pandas
S-03	Los ~20 archivos de eventos tienen la misma estructura y distribución que el perfilado	Cambia el dimensionamiento del watermark (D-008) y posiblemente el patrón (A-07)
S-04	Los nulos de resolved_at y csat son estado válido y no pérdida de datos	Habría que reclasificarlos como defectos y el mart de Soporte cambiaría de grano
S-05	La tasa de cambio ruidosa en USD es un defecto de datos y no una moneda mal etiquetada	Se revierte D-007; la evidencia en contra es que las tasas de USD se distribuyen simétricamente alrededor de 1,0
S-06	AstraDB está disponible como servicio gestionado para la instancia de serving	Habría que levantar Cassandra local en infra/ , con impacto en el Quickstart
S-07	Los identificadores duplicados hallados son duplicados de ingesta y no entidades distintas	La deduplicación en Bronze estaría borrando información legítima

11.2 Método de detección de anomalías

D-010 · MAD sobre la serie diaria por organización y servicio. Cierra A-04.

La consigna admite z-score, MAD o percentiles "con métodos justificados". Los tres se evaluaron sobre los datos reales antes de decidir.

Primer resultado: aplicados globalmente, los tres fallan.

Método	Eventos marcados	%
z-score z > 3	6	1,7%
z-score z > 2	20	5,6%
MAD mz > 3,5	86	23,9%
Percentiles (p01 / p99)	7	1,9%

Que MAD marque casi una cuarta parte de los eventos no es un error del método: es un síntoma. La distribución global es una mezcla de escalas incompatibles.

Métrica	n	Mediana de costo
cpu_hours	90	0,0594
storage_gb_hours	119	0,3515
requests	151	5,1741

Un evento de requests cuesta 87 veces la mediana de uno de cpu_hours . Cualquier estadística global marca como anómala la métrica cara simplemente por ser cara. Agrupar por servicio tampoco alcanza: cada servicio mezcla métricas y sus medianas quedan artificialmente parejas, entre 0,46 y 1,23.

La decisión. La detección no opera sobre eventos crudos sino sobre el mart diario org_daily_usage_by_service , donde la mezcla de métricas ya está colapsada en un único daily_cost_usd por celda. La línea de base se calcula por serie (org_id, service) a lo largo del tiempo: cada día se compara contra el comportamiento histórico de esa misma organización en ese mismo servicio.

```
mediana_serie = mediana( daily_cost_usd de (org, service) en los últimos N días )
MAD_serie     = mediana( |daily_cost_usd - mediana_serie| )
score         = 0,6745 × (daily_cost_usd - mediana_serie) / MAD_serie
is_anomaly    = |score| > 3,5
```

Por qué MAD y no z-score. El z-score usa media y desvío, y ambos se contaminan con los propios valores extremos que busca detectar: un spike infla el desvío y se vuelve menos detectable. La mediana y el MAD tienen punto de ruptura del 50%. Con una distribución asimétrica (media 2,59 contra mediana 0,75) la diferencia es material.

Por qué no percentiles solos. Un corte en p99 marca siempre el 1% de los casos, haya o no anomalías. Es una cuota, no una detección.

Viabilidad verificada. El mart tiene 181 series (org, service) con mediana de 2 días de historia en un archivo; extrapolado a los ~20 archivos, las 181 series superarían los 20 días, suficiente para una estadística robusta. Cuando una serie tenga menos de 20 días, se cae a la estadística agrupada por servicio y se marca el origen de la línea de base.

Los negativos se tratan aparte. Un costo de -19,82 no es un spike: es probablemente una nota de crédito, un fenómeno de negocio distinto. Se marca con is_credit_adjustment y no compite con la detección de picos, que quedaría sesgada si se mezclaran.

Reconocimiento del límite. Con 4 meses de datos no hay estacionalidad observable. Un método que descompusiera tendencia y estacionalidad sería superior, pero no se puede calibrar ni validar con esta ventana. Queda como próximo paso.

11.3 Riesgos

#	Riesgo	Prob.	Impacto	Mitigación	Señal de alerta
G-01	Los ~20 archivos no llegan a tiempo y el streaming no se puede ejercitar	Media	Alto	Generador sintético reproducible que replique el esquema v1/v2 y la fragmentación observada	No disponibles al 05/10
G-02	AstraDB inaccesible el día de la defensa	Media	Alto	Evidencias persistidas en evidence/ + plan alternativo con Cassandra local	Falla de conectividad en pruebas previas
G-03	El entorno efímero de Colab hace irreproducible la demo	Alta	Medio	Quickstart validado desde entorno limpio; datos de muestra reducidos	Una corrida completa supera los 15 minutos
G-04	La lógica duplicada del patrón híbrido diverge entre rutas	Media	Alto	Función de transformación compartida vía foreachBatch (§9.4); prueba que compara ambas salidas	Diferencias en el conteo entre streaming y reconciliación
G-05	El componente de ML (P7) no alcanza utilidad demostrable	Alta	Medio	Declarado deseable desde el inicio; la consigna admite alternativa equivalente acordada	Métricas sin poder predictivo en la primera prueba
G-06	Sobre-particionado genera miles de archivos triviales	Media	Bajo	coalesce(1) antes de escribir; evidencia de tamaños y conteo de archivos	Más de 100 archivos por zona en la demo
G-07	Alcance excesivo compromete el end-to-end	Media	Alto	Backlog priorizado en obligatorio / deseable / fuera de alcance tras la 2.ª entrega	Marts opcionales avanzando antes que el serving
G-08	El equipo subestima el esfuerzo de gobierno y documentación	Alta	Medio	Diccionario y DECISIONS.md se completan en paralelo, no al final	DECISIONS.md sin entradas nuevas en dos semanas

G-01 es el más accionable hoy y el único con fecha propia: si los archivos no están disponibles en dos semanas, conviene activar el generador en lugar de esperar.

11.4 Decisiones tomadas

Diez decisiones cerradas, registradas en DECISIONS.md con contexto, alternativa descartada y consecuencia.

#	Decisión	§
D-001	Cardinalidad exacta en el perfilado	4.1
D-002	Los rangos válidos se derivan del dominio observado	4.1
D-003	value se lee como String en la fuente de eventos	4.3
D-004	Patrón híbrido segmentado con reconciliación acotada	6.5
D-005	Bronze particiona por ingesta; Silver y Gold por evento	8.3
D-006	nps_surveys es la fuente de verdad del NPS	8.8
D-007	La tasa de cambio se fuerza a 1,0 para facturas en USD	8.8
D-008	Watermark de 7 días	9.5
D-009	El rechazo por calidad opera sobre el campo derivado	9.7
D-010	MAD sobre la serie diaria por organización y servicio	11.2

11.5 Decisiones abiertas

#	Tema	Bloquea	Cómo se resuelve
A-07	Verificar el orden cronológico en los ~20 archivos	D-004, D-008	Ejecutar el perfilado sobre el dataset completo. Si los eventos llegaran ordenados, el porcentaje descartado por el watermark caería y la reconciliación podría dejar de ser necesaria
A-08	Modelo de claves definitivo de las tablas de Cassandra	Serving	Se cierra al implementar, contrastando el CQL contra las cinco consultas obligatorias
A-09	Alcance final del componente analítico o de ML	P7	Se decide tras evaluar la señal disponible en la 2.ª entrega

A-07 es la única que condiciona una decisión ya tomada, y por eso es la primera de la lista. Las otras dos se resuelven naturalmente al implementar.

12. Estimación de esfuerzo, roles y recursos

12.1 Roles

El equipo es de **4 integrantes**. Cada rol tiene un responsable y un suplente: nadie queda como único punto de conocimiento sobre una parte del sistema, que es el riesgo real en un equipo chico con entregas encadenadas.

Rol	Responsabilidad	Artefactos que posee	Suplente
R1 · Arquitectura y gobierno	Coherencia entre diseño e implementación; decisiones y trade-offs; metadatos y linaje	Diagrama, DECISIONS.md , diccionario de datos, documento de diseño	R4
R2 · Ingesta y calidad	Lectores batch y streaming; Bronze; reglas de calidad y quarantine	Jobs de ingesta, esquemas explícitos, tests/ de reglas	R3
R3 · Procesamiento y analítica	Silver, features, marts Gold, detección de anomalías, componente de ML	Jobs de transformación, cost_anomaly_mart , análisis P7	R2
R4 · Serving y reproducibilidad	Modelado query-first, carga a Cassandra, Quickstart, infraestructura	Scripts CQL, config/ , infra/ , README.md	R1

El reparto no es por capas del pipeline sino por responsabilidad end-to-end de una capacidad. R2 no entrega "Bronze": entrega dato ingerido y validado, con sus pruebas. Eso evita la fricción clásica de que nadie se haga cargo de las fronteras.

Trabajo compartido por los cuatro: revisión cruzada antes de cada entrega, preparación de la defensa y actualización de DECISIONS.md cuando se toma una decisión, sin importar quién la tome.

12.2 Esfuerzo estimado

Estimación en horas-persona por instancia. El supuesto de capacidad es de 4 a 7 horas semanales por integrante, compatible con una materia electiva.

Instancia	Eje de trabajo	Horas-persona
1.ª entrega	Perfilado y evidencia	10
	Documento de diseño y diagrama	24
	Repositorio, convenciones y revisión	8
	Subtotal	42
2.ª entrega	Correcciones del feedback	12
	Ingesta batch a Bronze (3 maestros)	16
	Ingesta streaming con watermark y checkpoint	24
	Silver: conformación, v1/v2 y features	24
	Calidad y quarantine	14
	Mart FinOps en Gold	12
	Serving: keyspace, tabla y 2 consultas	18
	Idempotencia y evidencias	10
	Documentación y backlog	12
	Subtotal	142
Entrega final	Marts restantes (4 dominios)	22
	Anomalías y componente analítico	18
	Las 5 consultas y su evidencia	12
	Gobierno, linaje y observabilidad	14
	Pruebas, configuración y Quickstart validado	16
	Presentación, video y ensayo de defensa	20
	Subtotal	102
	Total del proyecto	286

Distribución en el tiempo. La segunda instancia concentra el 50% del esfuerzo en las 7 semanas entre el 28/09 y el 16/11: unas 5 horas semanales por integrante. La final concentra 102 horas en 3 semanas, unas 8,5 semanales por integrante — es el pico y conviene tenerlo presente al priorizar el backlog.

Fundamento de las cifras que más pesan. El streaming (24 h) y Silver (24 h) son los rubros más caros de la segunda entrega porque concentran los problemas que el perfilado documentó: watermark y deduplicación inter-lote en el primero, compatibilidad v1/v2 y conformación de siete fuentes en el segundo. La presentación y el video (20 h) suelen subestimarse y son requisito explícito de la instancia final.

12.3 Recursos

Recurso	Uso	Costo	Responsable
Google Colab	Ejecución de PySpark en modo local	Gratuito	R4
AstraDB	Keyspace y tablas de serving	Nivel gratuito	R4
Repositorio Git	Versionado, evidencias y revisión cruzada	Gratuito	R1
Almacenamiento compartido	Dataset completo (~20 archivos) fuera del repo	Gratuito	R2
PySpark 4.2.0 + Java 17+	Motor de procesamiento	—	R2

Ninguna credencial se versiona. Las de AstraDB se manejan por variable de entorno, con un archivo de ejemplo sin valores reales en `config/`.

12.4 Cadencia de trabajo

- **Sincronización semanal** de 30 minutos: estado, bloqueos y decisiones a tomar.
- **Revisión cruzada obligatoria** antes de cada entrega: cada rol revisa el trabajo de su suplente.
- **DECISIONS.md se actualiza al tomar la decisión**, no antes de entregar. El riesgo G-08 se materializa exactamente cuando se pospone.
- **Definición de terminado** para cualquier componente: corre desde entorno limpio, tiene evidencia en `evidence/`, y su decisión está registrada.

12.5 Trabajo restante para esta entrega

Tarea	Responsable	Estado
Perfilado y evidencia de exploración	R2	✓
Secciones 2 a 11 del documento	R1	✓
Diagrama de arquitectura v1	R1	✓
Repositorio, README y DECISIONS.md	R4	✓
Resumen ejecutivo	R1	pendiente
Obtener los ~20 archivos de eventos (G-01)	R2	pendiente, bloquea A-07
Exportar el documento a PDF	R1	pendiente
Verificación contra el checklist 9.1 de la consigna	los 4	pendiente

Después del 28/09. La jornada de clase es de feedback, y la consigna exige versionar un **plan de correcciones** con prioridad, responsable, fecha objetivo y evidencia esperada. Ese plan alimenta la arquitectura actualizada de la segunda entrega y se registra en `docs/`.

12.6 Próximos pasos más allá del alcance

Identificados durante el diseño y declarados como fuera de alcance:

- **Formato de lakehouse** (Delta o Iceberg): resolvería con transacciones ACID, evolución de esquema gestionada y time travel los tres problemas que hoy se atienden a mano.
- **Orquestación** con Airflow: hoy la cadencia de los flujos es manual.
- **Detección de anomalías con estacionalidad**: requiere una ventana histórica mayor a los 4 meses disponibles (§11.2).

- **Reconciliación incremental:** acotar la recomputación a las particiones tocadas en lugar de recalcular la ventana completa.

13. Anexo — Evidencia de exploración de datos

La evidencia completa del perfilado está en:

- `notebooks/01_perfilado_fuentes.py` — script reproducible en PySpark
- `evidence/perfilado_landing.md` — informe generado, con todas las tablas por fuente
- `evidence/perfilado_metrics.json` — métricas estructuradas, reutilizables en etapas posteriores

Reproducción desde un entorno limpio:

```
pip install pyspark
python notebooks/01_perfilado_fuentes.py
```

El script no escribe en Landing y puede re-ejecutarse sin efectos colaterales.